AI VIET NAM
@aivietnam.edu.vn

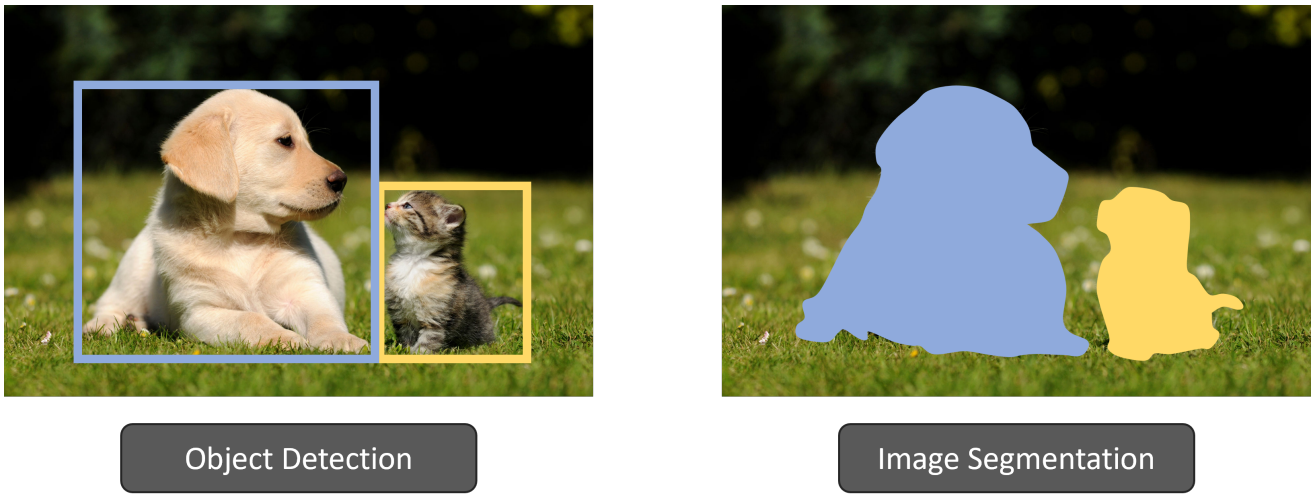
AI VIET NAM – AI COURSE 2025

Tutorial: Mô hình UNet và bài toán Segmentation

Văn Đình Tuệ Thư Trần Hoàng Duy Đinh Quang Vinh

I. Dẫn nhập

Image Segmentation là một nhiệm vụ quan trọng trong lĩnh vực thị giác máy tính, trong đó một bức ảnh được chia thành nhiều vùng khác nhau với mục tiêu đơn giản hóa hoặc thay đổi biểu diễn của bức ảnh thành một dạng có ý nghĩa hơn để dễ dàng phân tích. Image Segmentation thường được sử dụng để xác định vị trí các đối tượng hoặc đường biên, hay nói cách khác, đây là quá trình gán nhãn cho từng pixel trong ảnh, sao cho các pixel thuộc cùng một đối tượng hoặc khu vực sẽ được nhóm lại với nhau.

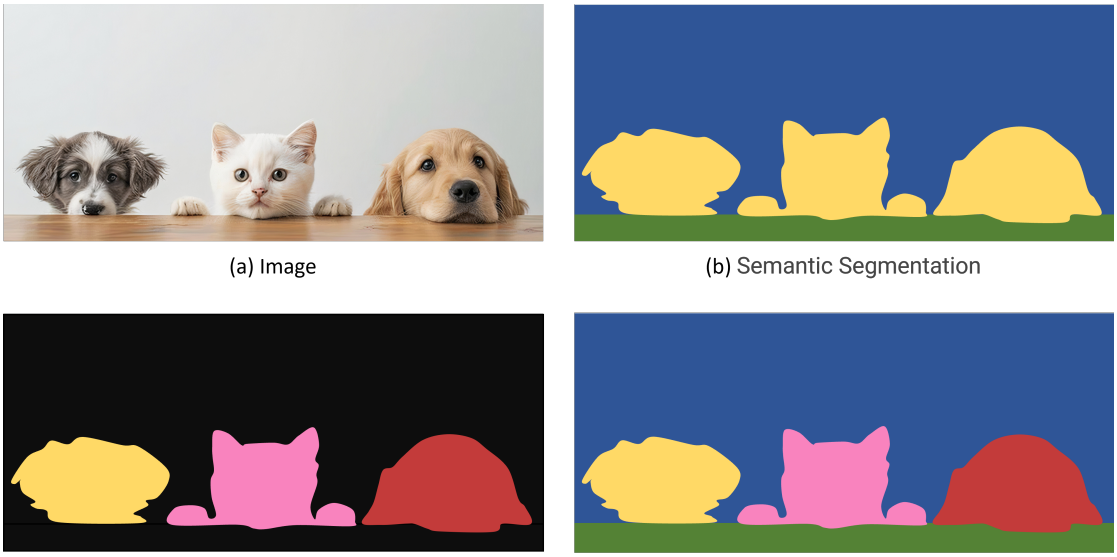


Hình 1: Minh họa phân biệt giữa Object Detection và Image Segmentation.

Image Segmentation và Object Detection đều có chung mục tiêu là xác định các vùng chứa đối tượng và gán nhãn cho chúng; tuy nhiên, Image Segmentation yêu cầu độ chính xác cao hơn. Khác với Object Detection, nơi đối tượng được định vị thông qua các hộp giới hạn (bounding boxes), Image Segmentation yêu cầu nhãn dự báo phải chính xác đến từng pixel trong ảnh. Điều này giúp chúng ta hiểu rõ hơn về nội dung bức ảnh, không chỉ dừng lại ở việc xác định vị trí các vật thể mà còn mô tả được hình dạng của chúng, cũng như gán từng pixel vào đúng vật thể tương ứng.

Image Segmentation được chia làm ba loại:

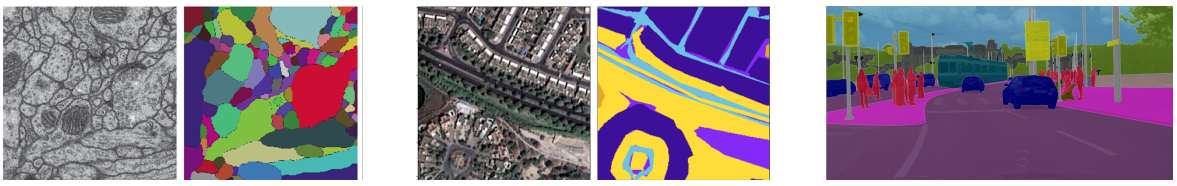
- **Semantic Segmentation**: Gán nhãn cho từng pixel trong ảnh và nhóm các pixel có cùng ngữ nghĩa vào một lớp cụ thể. Quá trình này phân đoạn các vùng ảnh theo từng lớp khác nhau mà không phân biệt giữa các đối tượng riêng lẻ thuộc cùng một lớp.
- **Instance Segmentation**: Phân đoạn các vùng ảnh chi tiết đến từng đối tượng trong mỗi lớp.
- **Panoptic segmentation**: Kết hợp giữa Semantic Segmentation và Instance Segmentation, trong đó mỗi đối tượng được phân đoạn với ranh giới rõ ràng, đồng thời danh tính và ý nghĩa của từng đối tượng trong ảnh được dự đoán chính xác.



Hình 2: Minh họa Semantic Segmentation, Instance Segmentation và Panoptic segmentation.

Image Segmentation thường được sử dụng trong nhiều ứng dụng thực tiễn như:

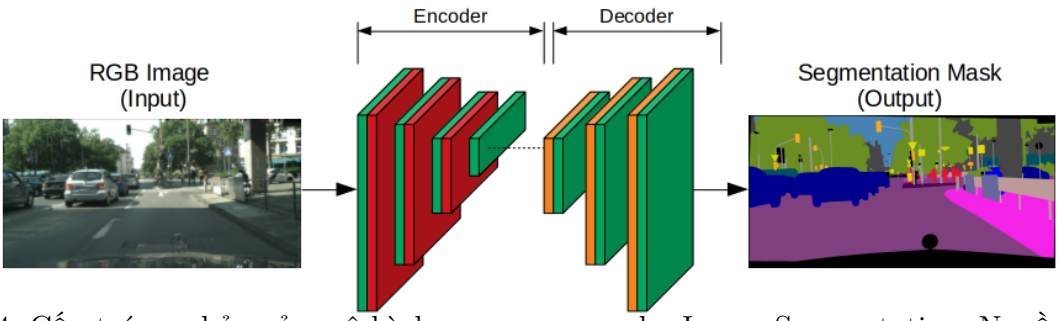
- Y học: Phân đoạn các tế bào, mô hoặc cơ quan trong ảnh chụp X-quang, CT, hoặc MRI.
- Xử lý ảnh vệ tinh: Phân loại vùng đất, rừng, sông ngòi từ các bức ảnh vệ tinh.
- Xe tự hành: Nhận diện đường, xe cộ, người đi bộ và các vật thể khác để hỗ trợ định vị và dẫn đường.



Hình 3: Minh họa ứng dụng của Image Segmentation trong (1) Y học, (2) Xử lý ảnh vệ tinh, (3) Xe tự hành.

Có hai phương pháp chính để thực hiện Image Segmentation:

- **Phương pháp truyền thống** - dựa trên các thuật toán như:
  - Phân ngưỡng (Thresholding) để chia điểm ảnh thành các lớp dựa trên cường độ.
  - Phân cụm (Clustering) như K-means để nhóm các điểm ảnh có đặc trưng tương đồng.
  - Phân đoạn theo vùng (Region-based Segmentation) dựa vào sự tương đồng về màu sắc hoặc kết cấu.
  - Phân đoạn theo cạnh (Edge-based Segmentation) để xác định ranh giới của các đối tượng trong hình ảnh.
- **Phương pháp hiện đại**:
  - Sử dụng mạng nơ-ron tích chập (CNN) để học và trích xuất các đặc trưng trực tiếp từ dữ liệu, được huấn luyện để tự động nhận diện những đặc trưng quan trọng trong ảnh, thay vì dựa vào các hàm tùy chỉnh như trong các phương pháp truyền thống.
  - Các mạng nơ-ron dùng để phân đoạn thường áp dụng cấu trúc encoder-decoder. Encoder sẽ trích xuất các đặc trưng của ảnh qua các bộ lọc ngày càng hẹp và sâu hơn. Nếu encoder được huấn luyện sẵn trên các nhiệm vụ như nhận diện khuôn mặt hay nhận dạng ảnh, nó sẽ tận dụng kiến thức đã học để hỗ trợ phân đoạn (học chuyển giao). Sau đó, decoder sẽ mở rộng kết quả của encoder qua nhiều lớp để tạo ra mặt nạ phân đoạn có độ phân giải pixel giống hệt ảnh đầu vào.



Hình 4: Cấu trúc cơ bản của mô hình mạng nơ-ron cho Image Segmentation. Nguồn: Link

Trong bối cảnh các bài toán phức tạp, các phương pháp học sâu đã trở thành lựa chọn hàng đầu nhờ khả năng xử lý chi tiết và duy trì thông tin không gian. Đặc biệt, kiến trúc mạng U-Net nổi bật với thiết kế đối xứng độc đáo và khả năng phân đoạn chính xác ở cấp độ pixel, giúp bảo tồn các chi tiết nhỏ trong ảnh. Do đó, U-Net đã trở thành giải pháp lý tưởng cho những bài toán đòi hỏi độ chi tiết cao, nơi việc duy trì thông tin không gian là vô cùng quan trọng.

Trong tài liệu này, chúng ta sẽ khám phá kiến trúc mạng U-Net và cách áp dụng nó để giải quyết bài toán về Image Segmentation.

Mục lục

I. Dẫn nhập **1**

II. Mạng U-Net **5**

II.1. Skip Connections 6

II.2. Transposed Convolution 6

II.3. Encoder 10

II.4. Decoder 11

III. Thực Hành **12**

III.1. Bài toán 1: Segmentation cho bộ dữ liệu chó mèo 12

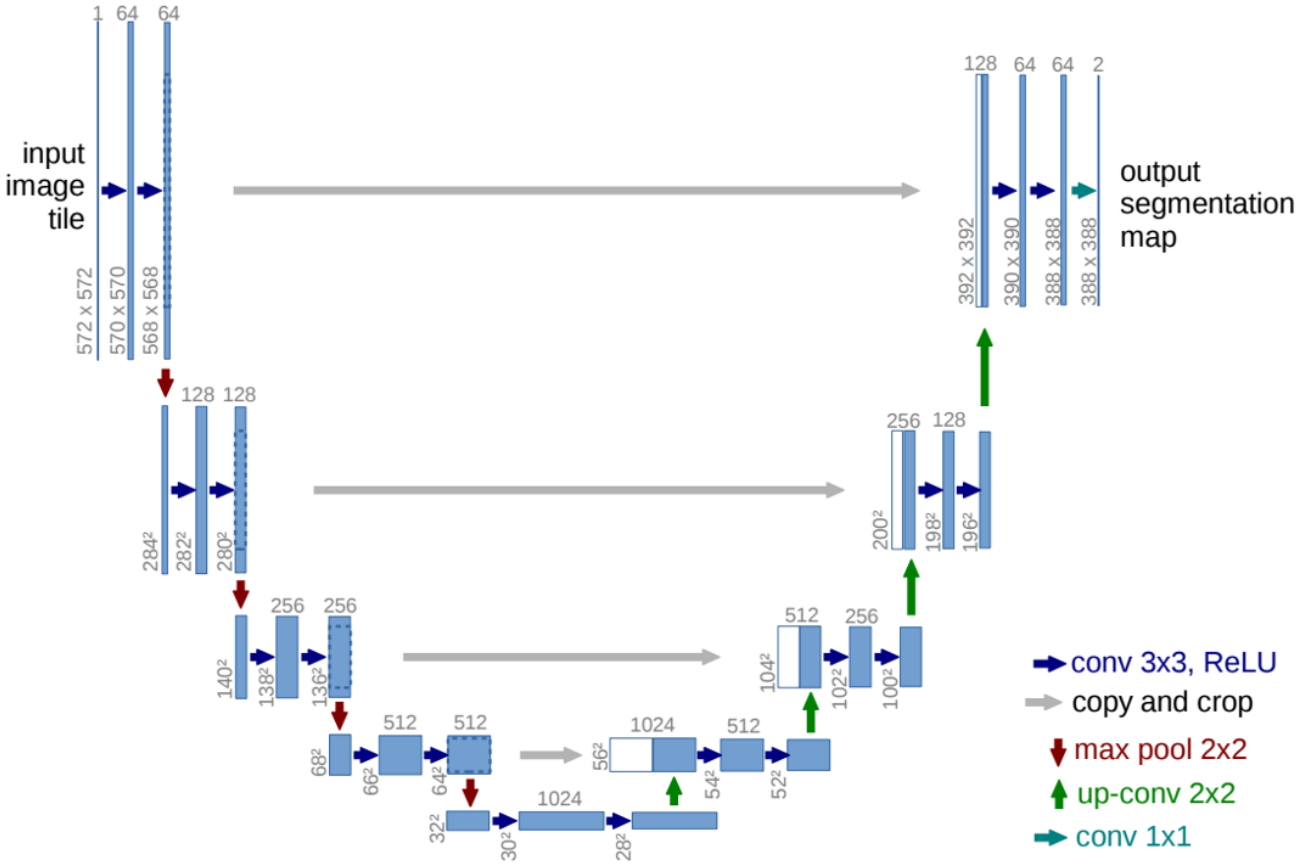
III.2. Bài toán 2: Segmentation cho Kvasir-SEG dataset 19

IV. Tài liệu tham khảo **26**

Phụ lục **26**

II. Mạng U-Net

U-Net là một mạng nơ-ron tích chập được thiết kế ban đầu cho phân đoạn ảnh y sinh. Kiến trúc của U-Net bao gồm 2 phần chính là encoder và decoder đối xứng nhau với hình dạng giống chữ U.



Hình 5: Kiến trúc mạng U-Net. Nguồn: Link

Hình 5 trên mô tả kiến trúc mạng U-Net, trong đó:

- Mỗi hình chữ nhật màu xanh biểu thị một multi-channel feature map.
- Số lượng channels được ghi chú ở phía trên hình chữ nhật.
- Kích thước x-y được cung cấp ở góc dưới bên trái của hình chữ nhật.
- Các hình chữ nhật màu trắng đại diện cho các feature maps được sao chép.
- Các mũi tên biểu thị các operations khác nhau:

Mũi tên	Ý nghĩa
	Conv layer 3x3, ReLU
	Skip Connections (có crop)
	Max Pooling 2x2
	Transposed Convolution
	Conv layer 1x1

Bảng 1: Ý nghĩa các mũi tên trong kiến trúc mạng U-Net.

II.1. Skip Connections

U-Net lần đầu tiên giới thiệu skip connections trong Deep Learning như một giải pháp hiệu quả để khắc phục mất mát thông tin trong quá trình downsampling, thường gặp trong các khối mã hóa (encoder) của kiến trúc encoder-decoder. Skip connections là các kết nối trực tiếp từ encoder đến decoder mà không cần đi qua bottleneck, giúp giảm thiểu mất mát dữ liệu do các phương pháp aggressive pooling và downsampling gây ra.

Việc sử dụng skip connections giúp khắc phục vấn đề mất thông tin ở spatial dimension. Nếu chỉ dựa vào feature map từ bottleneck để xây dựng lại feature map có độ phân giải cao, nhiều chi tiết về vị trí pixel sẽ bị mất. Nhờ các kết nối này, thông tin từ phần encoder được truyền trực tiếp sang decoder, hỗ trợ việc tái tạo vị trí pixel chính xác hơn.

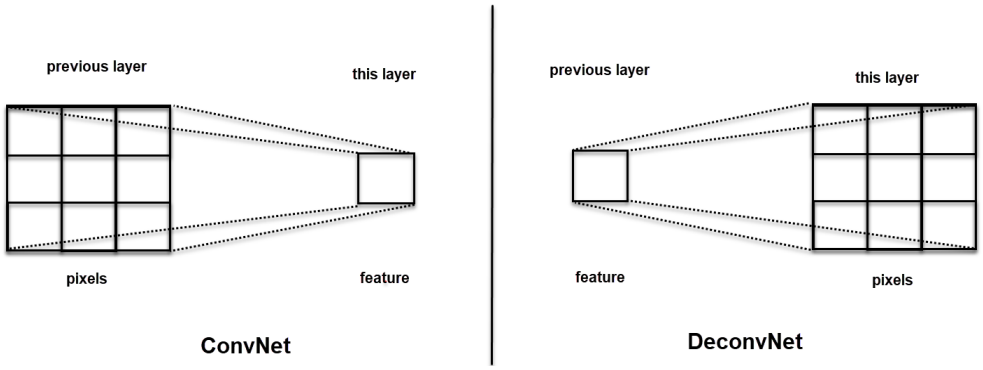
II.2. Transposed Convolution

Convolution là một phép toán cơ bản trong xử lý dữ liệu, đặc biệt là trong mạng CNN. Phép toán này sử dụng một kernel trượt qua dữ liệu đầu vào, tính tích chập để tạo ra feature map. Kết quả thường có kích thước nhỏ hơn, giúp trích xuất các đặc trưng cục bộ quan trọng như cạnh, góc hoặc mẫu hình.

Trong khi Convolution giảm kích thước không gian của dữ liệu, Transposed Convolution (hay còn gọi là Deconvolution) thực hiện ngược lại: nó mở rộng dữ liệu đầu vào. Đây là một kỹ thuật phổ biến trong các bài toán như Image Generation hoặc Image Segmentation, nơi cần khôi phục dữ liệu về kích thước ban đầu hoặc tạo dữ liệu có độ phân giải cao hơn.

Trong Convolution tiêu chuẩn, kích thước không gian của feature map được giảm bằng cách áp dụng kernel có kích thước cố định, kết hợp với stride và padding. Ngược lại, Transposed Convolution thực hiện chức năng mở rộng kích thước không gian của feature map bằng cách áp dụng kernel lên đầu vào, đồng thời điều chỉnh kích thước output thông qua stride và padding. Có thể hiểu, Transposed Convolution là quá trình ngược của Convolution thông thường, trong đó mỗi feature được ánh xạ thành các pixel của ảnh, thay vì ánh xạ các pixel thành feature như trong Convolution tiêu chuẩn (hình 6 dưới đây minh họa sự khác biệt giữa hai quá trình này).

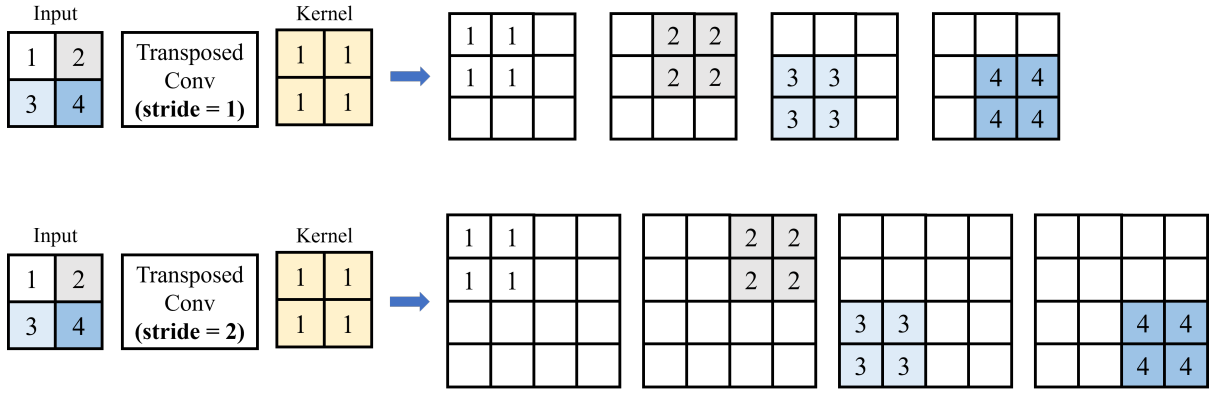
So với các phương pháp upsampling truyền thống như Nearest Neighbors, Bi-Linear Interpolation hay Max-Unpooling, Transposed Convolution vượt trội hơn nhờ khả năng học các tham số từ dữ liệu, giúp khôi phục thông tin không gian một cách chi tiết và chính xác hơn.



Hình 6: So sánh Convolution với Transposed Convolution. Nguồn: Link

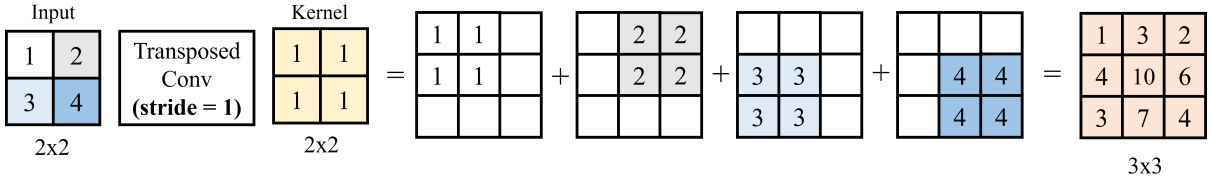
Quy trình thực hiện Transposed Convolution:

1. Nhân tích chập và xử lý dịch chuyển stride:
   - Mỗi phần tử trong input feature map được nhân với kernel, sau đó kết quả được đặt vào vị trí tương ứng trên output feature map.
   - Với stride = s, kết quả sẽ dịch chuyển s pixel sau mỗi lần tính toán, và stride lớn sẽ làm tăng kích thước của output.



Hình 7: Minh họa bước nhân tích chập và xử lý dịch chuyển stride với hai giá trị stride bằng 1 và 2.

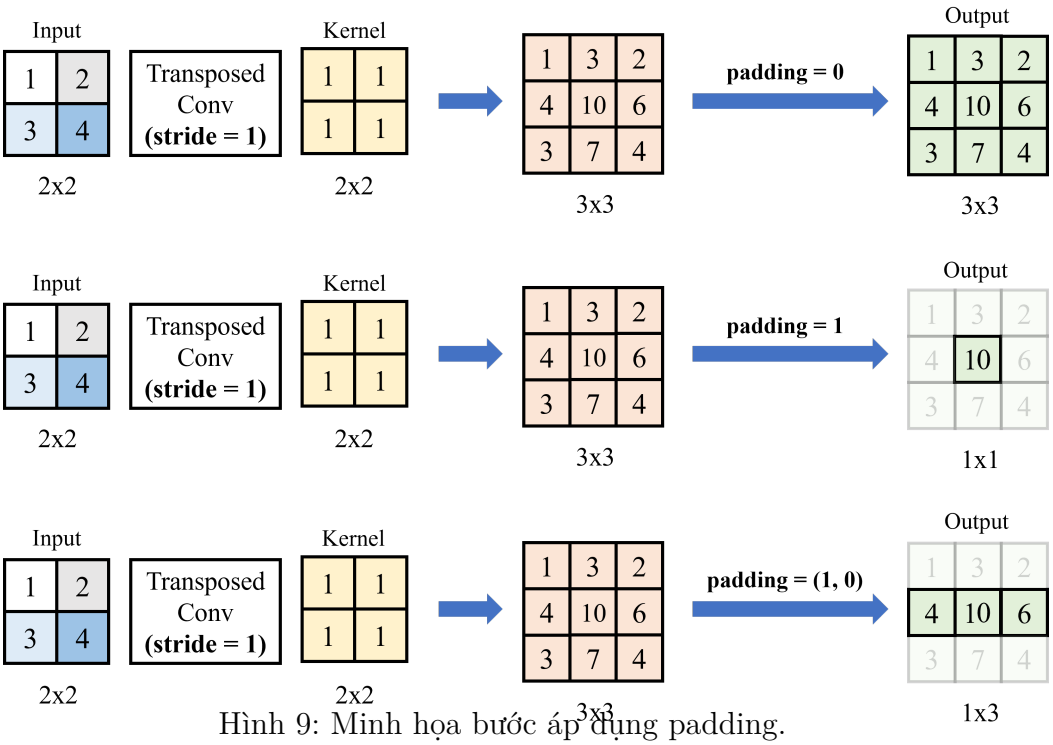
2. Xử lý chồng chéo (overlap): Khi kích thước stride nhỏ hơn kích thước kernel, các vùng trên ma trận kết quả sẽ bị chồng lên nhau. Trong trường hợp này, các giá trị tại các vị trí chồng chéo sẽ được cộng lại để tạo thành giá trị cuối cùng tại vị trí đó.



Hình 8: Minh họa bước xử lý chồng chéo (overlap).

3. Áp dụng padding:

   - Padding được thêm vào để đảm bảo kích thước output phù hợp.
   - Với padding = p, tiến hành loại bỏ p hàng và cột ở 4 cạnh (trên, dưới, trái, phải) của kết quả đầu ra. Lưu ý:
     - **Khi padding = (p, q)**: bỏ đi p hàng đầu tiên và cuối cùng (trên, dưới); q cột đầu tiên và cuối cùng (trái, phải) của output.
     - **Khi padding = p**: đây là cách viết rút gọn, tương đương với padding = (p, p).



Hình 9: Minh họa bước áp dụng padding.

   - Công thức tổng quát để xác định kích thước output:

$$Height : H_{\text{out}} = (m-1) \cdot s - 2p + (h-1) + 1$$

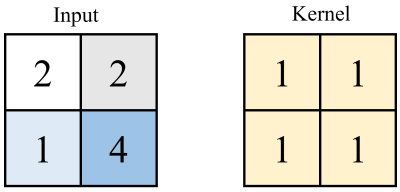
$$Width : W_{\text{out}} = (n-1) \cdot t - 2q + (k-1) + 1$$

     Trong đó:
     - m x n: kích thước input.
     - h x k: kích thước kernel.
     - s x t: kích thước stride.
     - p x q: kích thước padding.

Ví dụ minh họa:

Để hiểu rõ hơn các bước tính toán trong quá trình Transposed Convolution, chúng ta sẽ cùng thực hiện hai ví dụ với input và kernel được mô tả trong Hình 10.

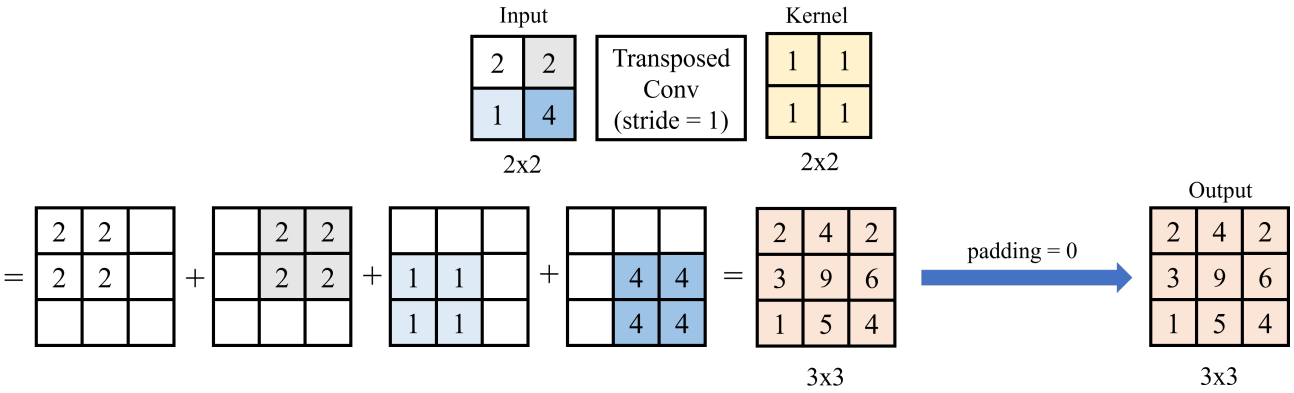
- Input: Một encoded feature map kích thước 2x2.
- Kernel có kích thước 2x2.



Hình 10: Giá trị Input và Kernel cho hai ví dụ tính toán quá trình Transposed Convolution.

Ví dụ 1: Với stride = 1 và padding = 0.

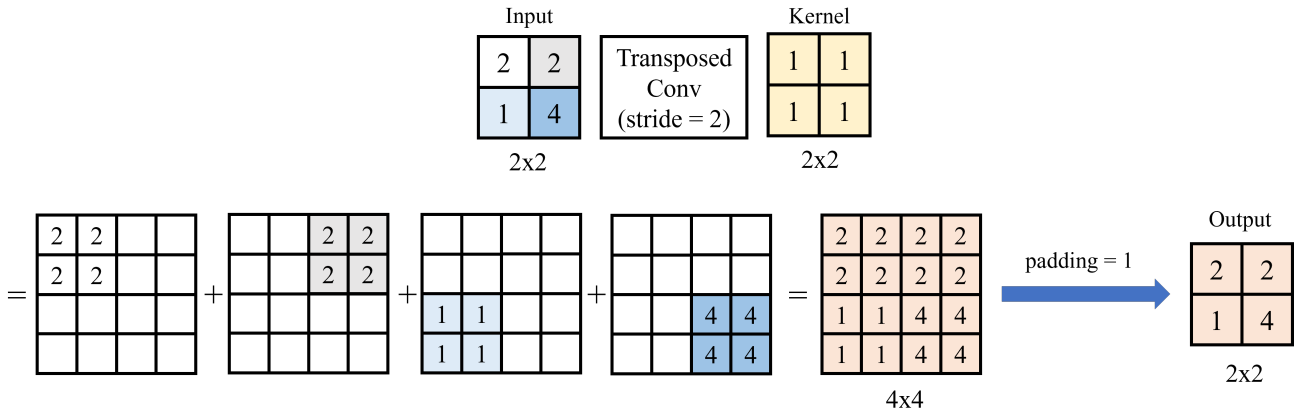
1. Đầu tiên, lấy từng pixel input nhân với kernel (nhân với từng phần tử). Với stride = 1, di chuyển kết quả của mỗi lần nhân pixel với kernel sang 1 pixel.
2. Sau khi thực hiện nhân tích chập, vì stride có kích thước nhỏ hơn kernel nên cần cộng các giá trị chồng chéo (overlap) lại với nhau. Trong trường hợp này, vì padding = 0 nên không loại bỏ hàng hay cột nào ở output. Hình 11 dưới đây minh họa cho quá trình tính Transposed Convolution cho ví dụ đầu tiên:



Hình 11: Minh họa các bước tính Transposed Convolution cho ví dụ 1.

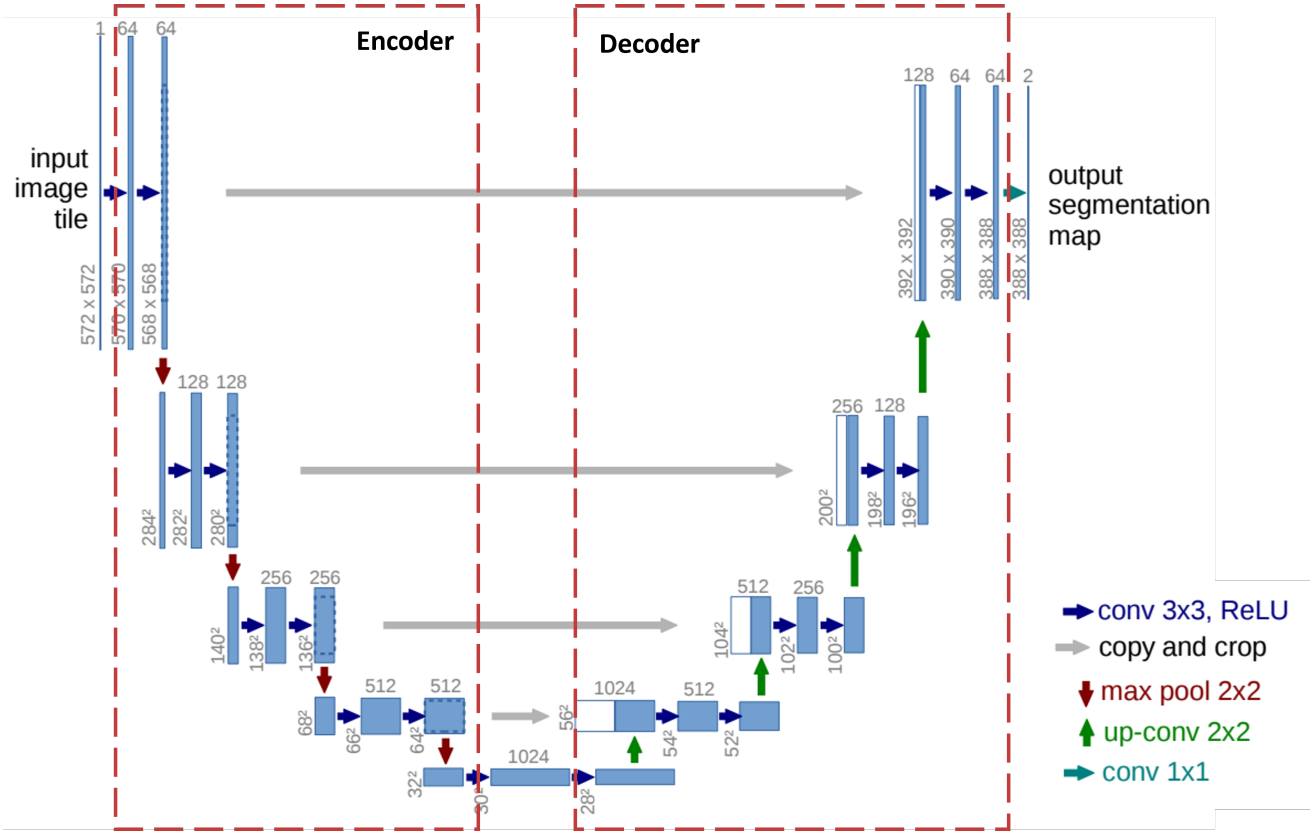
Ví dụ 2: Với stride = 2 và padding = 1.

1. Đầu tiên, lấy từng pixel input nhân với kernel (nhân với từng phần tử). Với stride = 2, di chuyển kết quả của mỗi lần nhân pixel với kernel sang 2 pixel.
2. Sau khi thực hiện nhân chập, các giá trị không bị overlap vì kích trước của stride không nhỏ hơn kích thước kernel.
3. Với padding = 1, tiến hành loại bỏ 1 hàng và cột ở 4 cạnh (trên, dưới, trái, phải) của kết quả đầu ra. Hình 12 dưới đây minh họa trình tính Transposed Convolution cho ví dụ 2:



Hình 12: Minh họa các bước tính Transposed Convolution cho ví dụ 2.

II.3. Encoder



Hình 13: Kiến trúc U-net với 2 phần chính Encoder-Decoder.

Dựa vào hình 13, phần encoder bao gồm các lớp Conv và MaxPooling thông thường. Khi đi từ trên xuống dưới, kích thước width x height giảm, trong khi depth tăng lên (depth của output mỗi lớp được ghi ở trên đỉnh của hình chữ nhật, còn kích thước width và height được ghi dọc theo hình chữ nhật).

Encoder được thiết kế với mục tiêu trích xuất các đặc trưng quan trọng từ ảnh đầu vào thông qua các lớp convolution và pooling. Quá trình này không chỉ giúp nhận diện các đặc trưng quan trọng mà còn giảm dần kích thước không gian của ảnh, đồng thời giữ lại những thông tin đặc trưng cần thiết cho các bước xử lý tiếp theo trong mô hình.

II.4. Decoder

Phần này được thiết kế ngược lại với encoder vì nó làm tăng kích thước width x height, đồng thời giảm depth. Để thực hiện điều này, ta sử dụng Transposed Convolution. Tại mỗi giai đoạn của decoder, layer đối xứng tương ứng của encoder sẽ được crop và concatenate lại. Từ hình 13, ta có thể thấy rõ cách thức hoạt động của decoder, với các bước thực hiện tăng dần kích thước không gian và tái tạo lại thông tin đã bị nén trong encoder.

Khác với các mô hình phân loại ảnh truyền thống sử dụng lớp fully connected (FC) ở cuối cùng, U-Net thực hiện dự đoán trực tiếp thông qua phần up-sampling của decoder để tạo ra một bản đồ phân loại có kích thước pixel tương ứng với ảnh đầu vào. Vì đây là bài toán phân lớp cho từng điểm ảnh (pixel-wise classification), hàm mất mát (loss function) thường được sử dụng là tổng Cross Entropy tính trên toàn bộ các pixels. Việc tối ưu hóa hàm loss này cho phép mô hình học được cách gán nhãn chính xác cho từng vùng đối tượng, đảm bảo độ chi tiết cao cho mặt nạ phân đoạn cuối cùng.

III. Thực Hành

III.1. Bài toán 1: Segmentation cho bộ dữ liệu chó mèo

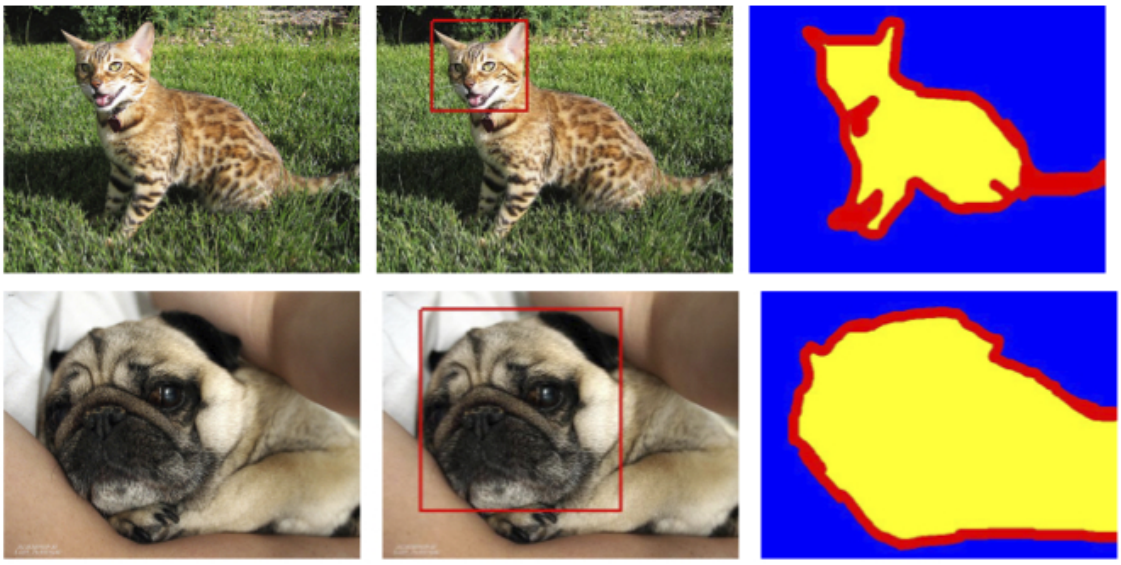
Trong phần thực hành, chúng ta sẽ sử dụng kiến trúc U-Net để giải quyết bài toán Image Segmentation trên các bộ dataset cụ thể. Quá trình sẽ bao gồm việc huấn luyện mô hình và đánh giá hiệu suất trên dữ liệu thực tế.

Đối với bài toán này, chúng ta sẽ xây dựng một mô hình Segmentation nhằm phân đoạn hình ảnh thú cưng (chó/mèo) từ tập dữ liệu **Oxford-IIIT Pet Dataset**. Mục tiêu của bài toán là tạo ra một mặt nạ (mask) phân đoạn, trong đó mỗi pixel được gán vào một trong ba lớp: **Background** (nền), **Pet** (vùng chứa thú cưng), **Contour**(đường viền phân tách giữa thú cưng và nền).

Bài toán segmentation này không chỉ giúp xác định rõ ràng vùng thú cưng trong ảnh mà còn là nền tảng để giải quyết các bài toán phức tạp hơn, như tách nền tự động, phân tích hình dáng, hoặc nhận diện giống loài.

III.1.1. Import và tiền xử lý dữ liệu

Oxford-IIIT Pet Dataset là một tập dữ liệu phổ biến, được thiết kế dành cho các bài toán **Image Classification** và **Image Segmentation**. Bộ dữ liệu này do nhóm nghiên cứu tại Đại học Oxford phát triển, cung cấp cả nhãn phân loại chó mèo và mặt nạ phân đoạn chi tiết.



Hình 14: Một số mẫu dữ liệu trong Oxford-IIIT Pet Dataset.

Trong phần này, chúng ta sẽ xử lý dữ liệu từ **Oxford-IIIT Pet Dataset** bằng cách áp dụng các phép biến đổi (transforms) cho ảnh và mặt nạ phân đoạn. Ảnh sẽ được chuẩn hóa, resize về kích thước đồng nhất và chuyển đổi thành tensor, trong khi mặt nạ phân đoạn được xử lý để đảm bảo các giá trị pixel phù hợp với định dạng yêu cầu của mô hình.

Dữ liệu đã được chia sẵn thành hai phần: tập huấn luyện (**trainval**) và tập kiểm tra (**test**) để sử dụng cho quá trình huấn luyện và đánh giá mô hình một cách hiệu quả.

Tiền xử lý dữ liệu

```
import copy
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt

from torchvision import transforms
from torchvision.datasets import OxfordIIITPet
from torch.utils.data import DataLoader

img_size = (128, 128)
num_classes = 3

transform = transforms.Compose([transforms.Resize(img_size), transforms.ToTensor(),
              transforms.Normalize((0.485, 0.456, 0.406),
                                   (0.229, 0.224, 0.225))])

def target_transform(target):
    img = transforms.Resize(img_size)(target)
    img = transforms.functional.pil_to_tensor(img).squeeze_()
    img = img - 1

    img = img.to(torch.long)
    return img

train_set = OxfordIIITPet(root="pets_data", split="trainval",
                          target_types="segmentation",
                          transform=transform,
                          target_transform=target_transform,
                          download=True)

test_set = OxfordIIITPet(root="pets_data", split="test",
                         target_types="segmentation",
                         transform=transform,
                         target_transform=target_transform,
                         download=True)

batch_size = 64
train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True, num_workers=6
                                                                                      )
test_loader = DataLoader(test_set, batch_size=batch_size, num_workers=6)
```

Quá trình de-normalize được thực hiện để đưa dữ liệu về phạm vi giá trị ban đầu, giúp hiển thị ảnh rõ ràng hơn để có thể kiểm tra và đánh giá mô hình.

De-normalize

```python
def de_normalize(img, mean=(0.485, 0.456, 0.406), std=(0.229, 0.224, 0.225)):
    result = img * std + mean
    result = np.clip(result, 0.0, 1.0)
    return result
```

Thiết kế khối Convolutional (ConvBlock), đây là thành phần cốt lõi của U-Net. Khối này được sử dụng để trích xuất đặc trưng từ ảnh đầu vào thông qua các lớp tích chập. Sau đó, ConvBlock sẽ được sử dụng để xây dựng các thành phần chính trong U-Net như Encoder và Decoder.

Xây dựng khối Convolutional

Convolutional Block

```python
class ConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels) -> None:
        super().__init__()
        self.conv_block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False)
                                                ,
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True),)
    def forward(self, x):
        x = self.conv_block(x)
        return x
```

Xây dựng khối Encoder

Encoder Block

```python
class Encoder(nn.Module):
    def __init__(self, in_channels, out_channels) -> None:
        super().__init__()
        self.encoder = nn.Sequential(
            nn.MaxPool2d(2),
            ConvBlock(in_channels, out_channels))
    def forward(self, x):
        x = self.encoder(x)
        return x
```

Xây dựng khối Decoder

Decoder Block

```python
class Decoder(nn.Module):
    def __init__(self, in_channels, out_channels) -> None:
        super().__init__()
        self.conv_trans = nn.ConvTranspose2d(in_channels, out_channels, kernel_size=4,
                                             stride=2, padding=1)
        self.conv_block = ConvBlock(in_channels, out_channels)

    def forward(self, x1, x2):
        x1  = self.conv_trans(x1)
        x = torch.cat([x2, x1], dim=1)
        x = self.conv_block(x)
        return x
```

Xây dựng Model

Model Architecture

```python

class UNet(nn.Module):
    def __init__(self, n_channels, n_classes) -> None:
        super().__init__()
        self.n_channels = n_channels
        self.n_classes = n_classes

        self.in_conv = ConvBlock(n_channels, 64)

        self.enc_1 = Encoder(64, 128)
        self.enc_2 = Encoder(128, 256)
        self.enc_3 = Encoder(256, 512)
        self.enc_4 = Encoder(512, 1024)

        self.dec_1 = Decoder(1024, 512)
        self.dec_2 = Decoder(512, 256)
        self.dec_3 = Decoder(256, 128)
        self.dec_4 = Decoder(128, 64)

        self.out_conv = nn.Conv2d(64, n_classes, kernel_size=1)

    def forward(self, x):
        x1 = self.in_conv(x)

        x2 = self.enc_1(x1)
        x3 = self.enc_2(x2)
        x4 = self.enc_3(x3)
        x5 = self.enc_4(x4)

        x = self.dec_1(x5, x4)
```

```
        x = self.dec_2(x, x3)
        x = self.dec_3(x, x2)
        x = self.dec_4(x, x1)

        x = self.out_conv(x)

        return x
```

Xây dựng hàm để hiển thị kết quả dự đoán

Hàm dự đoán

```
@torch.inference_mode()
def display_prediction(model, image, target):
    model.eval()
    img = image[None,...].to(device)
    output = model(img)
    pred = torch.argmax(output, axis=1)

    plt.figure(figsize=(10, 5))

    plt.subplot(1,3,1)
    plt.axis('off')
    plt.title("Input Image")
    plt.imshow(de_normalize(image.numpy().transpose(1,2,0)))

    plt.subplot(1,3,2)
    plt.axis('off')
    plt.title("Prediction")
    plt.imshow(pred.cpu().squeeze())

    plt.subplot(1,3,3)
    plt.axis('off')
    plt.title("Ground Truth")
    plt.imshow(target)

    plt.show()
```

Xây dựng hàm evaluate

Hàm đánh giá kết quả

```
def evaluate(model, test_loader, criterion):
    model.eval()
    test_loss = 0.0
    with torch.no_grad():
        for inputs, labels in test_loader:
```

```
            inputs, labels = inputs.to(device), labels.to(device)

            outputs = model(inputs)
            loss = criterion(outputs, labels)

            test_loss += loss.item()

    test_loss = test_loss / len(test_loader)
    return test_loss
```

Khai báo các tham số cho Model

Khai báo các tham số mô hình

```
device = "cuda" if torch.cuda.is_available() else "cpu"
max_epoch = 30
LR = 0.001
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=LR)
```

Thực hiện huấn luyện mô hình U-Net để giải quyết bài toán phân đoạn ảnh. Sau mỗi epoch, kết quả đánh giá sẽ được in ra màn hình cùng với hình ảnh dự đoán và mặt nạ thực tế để dễ dàng theo dõi và trực quan hóa hiệu suất của mô hình.

Huấn luyện mô hình

```
test_index = 80
display_image = test_set[test_index][0]
display_target = test_set[test_index][1]

train_losses = []
test_losses = []
best_loss = float('inf')
best_model_wts = copy.deepcopy(model.state_dict())

model.to(device)
for epoch in range(max_epoch):
    model.train()
    running_loss = 0.0
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        running_loss += loss.item()
        loss.backward()
        optimizer.step()
```

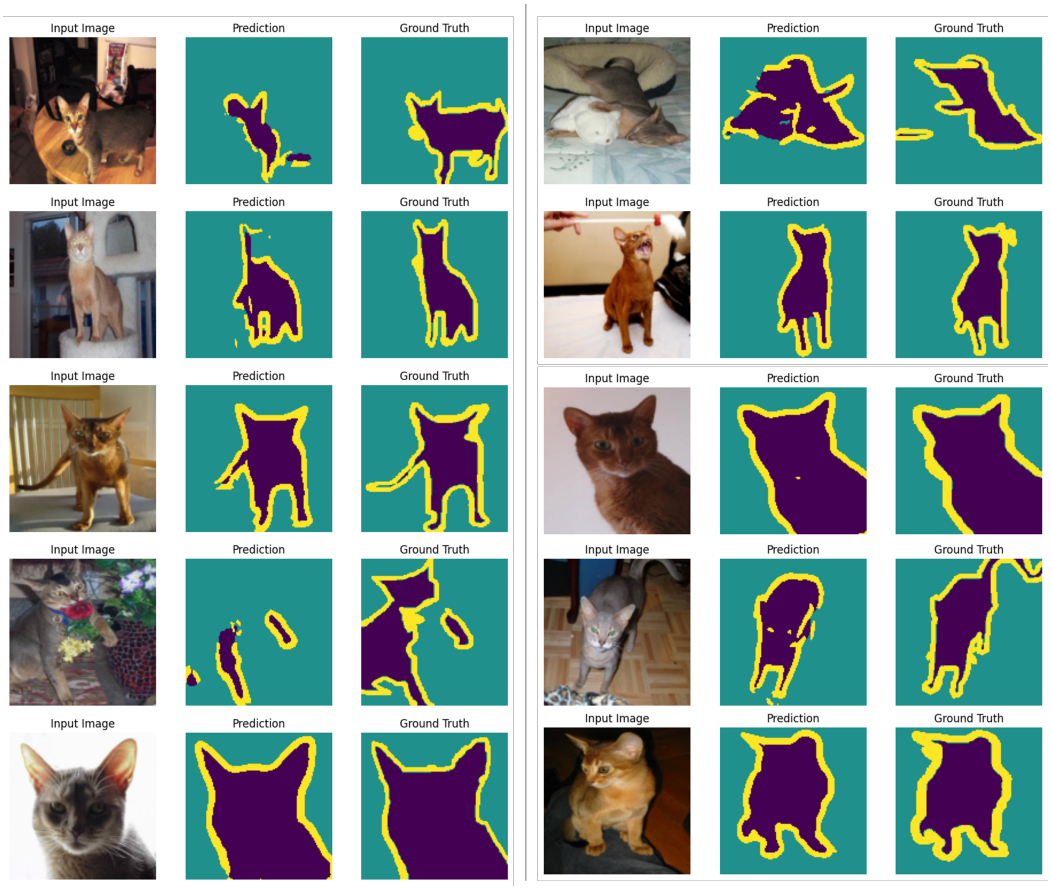
```
    epoch_loss = running_loss / len(train_loader)
    test_loss = evaluate(model, test_loader, criterion)
    if test_loss < best_loss:
        best_loss = test_loss
        best_model_wts = copy.deepcopy(model.state_dict())
    print(f"Epoch [{epoch + 1}/{max_epoch}], Trainning loss: {epoch_loss:.4f}, Test
                                        Loss: {test_loss:.4f}")
    print(f"Test image_{test_index} after epoch {epoch+1}: ")
    display_prediction(model, display_image, display_target)


train_losses.append(epoch_loss)
test_losses.append(test_loss)
```

Phần này là bước cuối cùng trong quy trình, mô hình thực hiện dự đoán 10 ảnh đầu tiên trên tập kiểm tra, giúp đánh giá hiệu suất mô hình và xác nhận rằng mô hình hoạt động tốt nhất với trọng số đã lưu.

Load trọng số mô hình và dự đoán

```
model.load_state_dict(best_model_wts)
n_test_points = 10
for i in range(n_test_points):
    img, gt = test_set[i]
    display_prediction(model, img, gt)
```

Sau khi huấn luyện xong, kết quả đạt được của mô hình như sau:



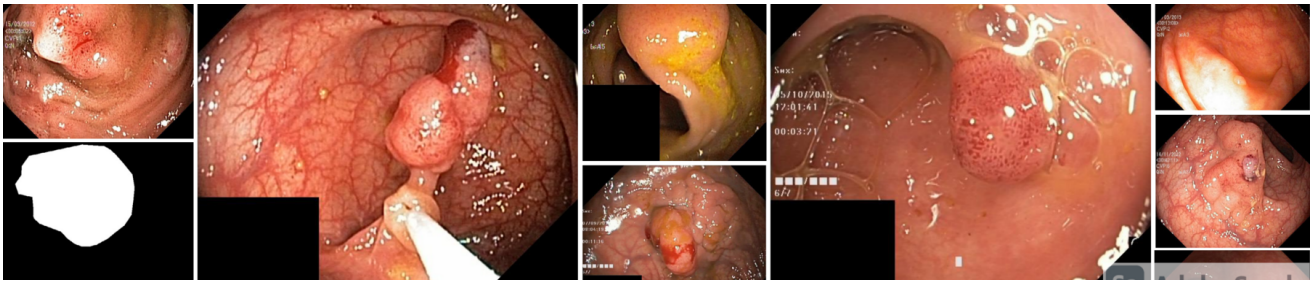
Hình 15: Kết quả dự đoán của mô hình sau khi huấn luyện.

III.2. Bài toán 2: Segmentation cho Kvasir-SEG dataset

Trong bài toán này, chúng ta sẽ xây dựng một mô hình Segmentation nhằm phân đoạn hình ảnh nội soi từ tập dữ liệu **Kvasir-SEG**. Mục tiêu chính của bài toán là tạo ra một mặt nạ (mask) phân đoạn, trong đó mỗi pixel được gán vào một trong hai lớp: **Background** (nền) hoặc **Polyp** (vùng tổn thương có hình dáng giống một khối u).

Import và Xử lý dữ liệu

Kvasir-SEG là một bộ dữ liệu chuẩn trong lĩnh vực phân đoạn y tế, đặc biệt được sử dụng cho các nhiệm vụ phân đoạn đa dạng như phát hiện và phân đoạn tổn thương trong nội soi tiêu hóa. Bộ dữ liệu này do Simula Research Laboratory phát triển và công bố, nhằm hỗ trợ nghiên cứu về phân tích hình ảnh nội soi.



Hình 16: Một số mẫu dữ liệu trong Kvasir-SEG dataset.

Tải bộ dữ liệu

```
!gdown 1ePUZYObyOefxWNqHVE5ibA_ADc_5Nqny
!unzip Kvasir-SEG.zip

ROOT_PATH = '/content/Kvasir-SEG/Kvasir-SEG'
images_path = os.path.join(ROOT_PATH, 'images')
masks_path = os.path.join(ROOT_PATH, 'masks')
```

Ở phần này chúng ta sẽ xử lý dữ liệu nhằm chuẩn bị đầu vào phù hợp, trong đó có việc chuyển đổi ảnh về ảnh xám (gray scale), với ảnh nội soi trong bộ dữ liệu Kvasir-SEG thường không yêu cầu thông tin màu sắc đầy đủ (RGB) để phân đoạn vùng tổn thương. Do đó, việc chuyển đổi sang ảnh xám là một cách tối ưu hóa tài nguyên, đồng thời giữ lại thông tin cần thiết phục vụ cho mô hình.

Tiền xử lý dữ liệu

```
import os
import cv2
import copy
import torch
import random
import numpy as np
from PIL import Image
from glob import glob
import torch.nn as nn
import torch.nn.functional
from torchvision import transforms
from matplotlib import pyplot as plt
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split

img_size = (128, 128)

image_transform = transforms.Compose([
    transforms.Resize(img_size),
    transforms.Grayscale(num_output_channels=1),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485], std=[0.229])
```

```
])

def mask_transform(mask):
    mask = transforms.Resize(img_size, interpolation=Image.NEAREST)(mask)
    mask = transforms.functional.pil_to_tensor(mask).squeeze(0)
    mask = mask / 255.0
    mask = torch.round(mask).long()
    return mask
```

Xây dựng các hàm tăng cường dữ liệu

Mặc dù bộ dữ liệu Kvasir-SEG cung cấp hình ảnh và nhãn chất lượng cao, số lượng mẫu trong tập dữ liệu này vẫn còn hạn chế (chỉ khoảng 1.000 ảnh). Điều này dễ dẫn đến overfitting, đặc biệt khi sử dụng các mô hình học sâu với số lượng tham số lớn.

Vì vậy, ta sẽ áp dụng các phương pháp data augmentation để củng cố lượng dữ liệu cho mô hình. Cụ thể:

1. **Elastic Transform (Biến dạng đàn hồi)**: Áp dụng biến dạng không gian mềm mại để mô phỏng sự thay đổi tự nhiên trong hình ảnh nội soi, giúp mô hình nhận diện đặc trưng tốt hơn khi hình ảnh bị biến dạng.
2. **Horizontal Flip (Lật ngang)**: Lật ảnh và mask theo chiều ngang, tăng khả năng nhận diện đặc trưng đối xứng, hữu ích với các tổn thương có tính đối xứng trái-phải.
3. **Vertical Flip (Lật dọc)**: Lật ảnh và mask theo chiều dọc, bổ sung sự đa dạng, giúp mô hình thích nghi với góc nhìn khác nhau.
4. **Combined Flip (Kết hợp lật và biến dạng đàn hồi)**: Kết hợp Elastic Transform, Horizontal Flip, và Vertical Flip để tạo các biến thể dữ liệu phong phú nhất, tăng khả năng học đặc trưng và cải thiện tổng quát hóa của mô hình.

Hàm tăng cường dữ liệu

```
def elastic_transform(image, mask, alpha_affine=10):
    random_state = np.random.RandomState(None)
    shape = image.size[::-1]  # (H, W)

    center_square = np.float32(shape) // 2
    square_size = min(shape) // 3
    pts1 = np.float32([center_square + square_size,
                       [center_square[0] + square_size, center_square[1] - square_size
                                                                                    ],
                       center_square - square_size])
    pts2 = pts1 + random_state.uniform(-alpha_affine, alpha_affine, size=pts1.shape).
                                                                    astype(np.float32)

    M = cv2.getAffineTransform(pts1, pts2)
    image = cv2.warpAffine(np.array(image), M, shape[::-1], borderMode=cv2.
```

```
                                            BORDER_REFLECT_101)
    mask = cv2.warpAffine(np.array(mask), M, shape[::-1], borderMode=cv2.
                                            BORDER_REFLECT_101)

    return Image.fromarray(image), Image.fromarray(mask)

def hflip_transform(image, mask):
    return image.transpose(Image.FLIP_LEFT_RIGHT), mask.transpose(Image.FLIP_LEFT_RIGHT
                                            )

def vflip_transform(image, mask):
    return image.transpose(Image.FLIP_TOP_BOTTOM), mask.transpose(Image.FLIP_TOP_BOTTOM
                                            )

def flip_transform(image, mask):
    return image.transpose(Image.FLIP_LEFT_RIGHT).transpose(Image.FLIP_TOP_BOTTOM),
                                            mask.transpose(Image.FLIP_LEFT_RIGHT).
                                            transpose(Image.FLIP_TOP_BOTTOM)
```

Xây dựng PyTorch Dataset

Pytorch Dataset

```
class KvasirDatasetAugmented(Dataset):
    def __init__(self, images_path, masks_path, transform=None, target_transform=None,
                                            augmentations=None):
        self.images_path = sorted(os.listdir(images_path))
        self.masks_path = sorted(os.listdir(masks_path))
        self.images_dir = images_path
        self.masks_dir = masks_path
        self.transform = transform
        self.target_transform = target_transform
        self.augmentations = augmentations if augmentations else []

        self.data = self._generate_augmented_data()

    def _generate_augmented_data(self):
        augmented_data = []
        for img_file, mask_file in zip(self.images_path, self.masks_path):
            img_path = os.path.join(self.images_dir, img_file)
            mask_path = os.path.join(self.masks_dir, mask_file)

            image = Image.open(img_path).convert("RGB")
            mask = Image.open(mask_path).convert("L")

            augmented_data.append((image, mask))

            for aug_func in self.augmentations:
                aug_image, aug_mask = aug_func(image, mask)
                augmented_data.append((aug_image, aug_mask))
```

```
        return augmented_data

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        image, mask = self.data[idx]

        if self.transform:
            image = self.transform(image)
        if self.target_transform:
            mask = self.target_transform(mask)

        return image, mask

# Khởi tạo dataset với Augmentation
augmentations = [elastic_transform, hflip_transform, vflip_transform, flip_transform]
dataset = KvasirDatasetAugmented(images_path, masks_path, transform=image_transform,
                                 target_transform=mask_transform,
                                 augmentations=augmentations)

# Phân chia dữ liệu
train_size = int(0.7 * len(dataset))
val_size = int(0.15 * len(dataset))
test_size = len(dataset) - train_size - val_size
train_dataset, val_dataset, test_dataset = torch.utils.data.random_split(dataset, [
                                 train_size, val_size, test_size])
```

Khai báo PyTorch Dataloader

Pytorch Dataloader

```
batch_size = 64

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True,
                          num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers=
                          2)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False,
                          num_workers=2)
```

Ở phần này chúng ta sẽ tiến hành xây dựng mô hình U-Net để Segmentation cho bộ dữ liệu Kvasir-SEG, nhưng vì cấu trúc mô hình không thay đổi nên chúng ta sẽ tái sử dụng cấu trúc mô hình U-Net từ bài toán trước.

Khai báo các tham số cho Model

Huấn luyện mô hình

```python
# Khai báo các tham số mô hình
device = "cuda" if torch.cuda.is_available() else "cpu"
model = UNet(n_channels=1, n_classes=2).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Huấn luyện mô hình
best_loss = float('inf')
best_model_wts = copy.deepcopy(model.state_dict())

for epoch in range(30):
    model.train()
    train_loss = 0.0
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(device), targets.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()
        train_loss += loss.item()
    train_loss = train_loss / len(train_loader)

    val_loss = evaluate(model, val_loader, criterion)

    if val_loss < best_loss:
        best_loss = val_loss
        best_model_wts = copy.deepcopy(model.state_dict())

    print(f"Epoch {epoch + 1}, Train Loss: {train_loss:.4f}, Val Loss: {val_loss:.4f}")
```

Xây dựng hàm để hiển thị kết quả dự đoán

Hàm dự đoán

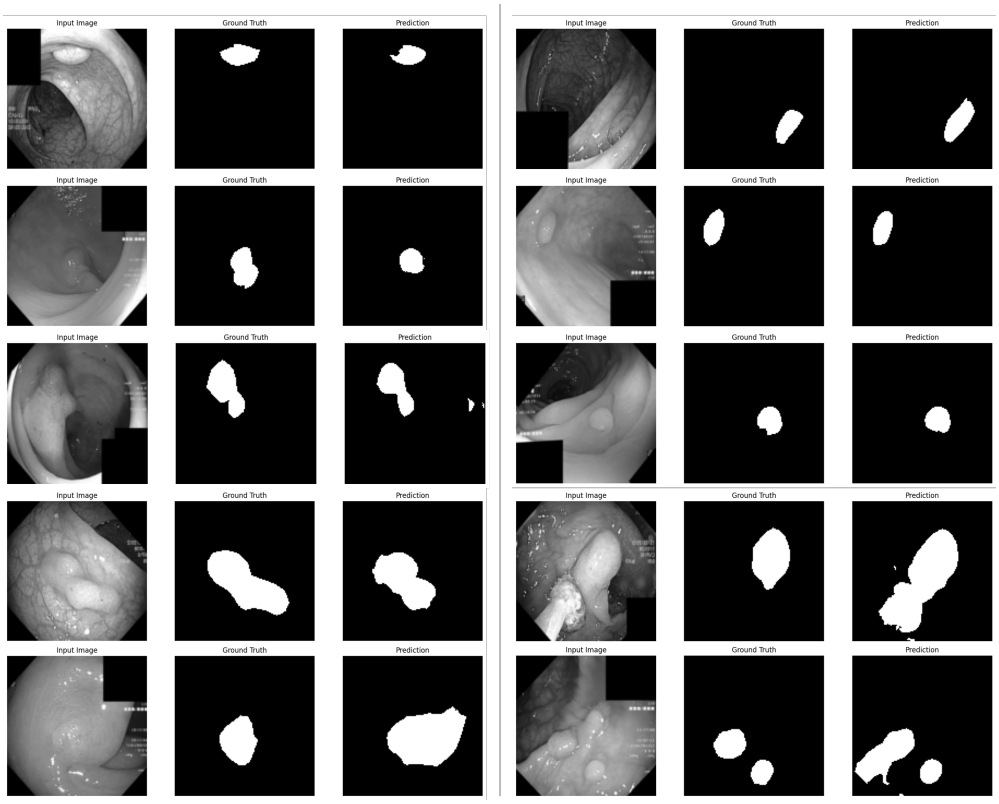
```python
def display_prediction(model, image, ground_truth, device):
    image = image.unsqueeze(0).to(device)
    with torch.no_grad():
        prediction = model(image)
        prediction = torch.argmax(prediction, dim=1).squeeze(0).cpu().numpy()

    image_np = image.squeeze(0).permute(1, 2, 0).cpu().numpy()
    ground_truth_np = ground_truth.cpu().numpy()

    fig, axs = plt.subplots(1, 3, figsize=(15, 5))
    axs[0].imshow(image_np, cmap='gray')
```

```
    axs[0].set_title("Input Image")
    axs[1].imshow(ground_truth_np, cmap='gray')
    axs[1].set_title("Ground Truth")
    axs[2].imshow(prediction, cmap='gray')
    axs[2].set_title("Prediction")
    for ax in axs:
        ax.axis("off")
    plt.show()
```

Mô hình được sử dụng để dự đoán 10 ảnh đầu tiên từ tập kiểm tra nhằm đánh giá hiệu suất tổng thể của mô hình và đảm bảo rằng nó đạt được kết quả tốt nhất với trọng số đã lưu.

Load trọng số mô hình và dự đoán

```
model.load_state_dict(best_model_wts)

n_test_points = 10
model.eval()

test_samples = [test_dataset[i] for i in range(n_test_points)]

for i in range(n_test_points):
    img, gt = test_samples[i]
    display_prediction(model, img, gt, device)
```



Hình 17: Kết quả dự đoán của mô hình sau khi huấn luyện.

IV. Tài liệu tham khảo

[1] O. Ronneberger et al., "U-net: Convolutional networks for biomedical image segmentation", in *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18*, Springer, 2015, pp. 234–241.

[2] J. Long et al., "Fully convolutional networks for semantic segmentation", in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 3431–3440.

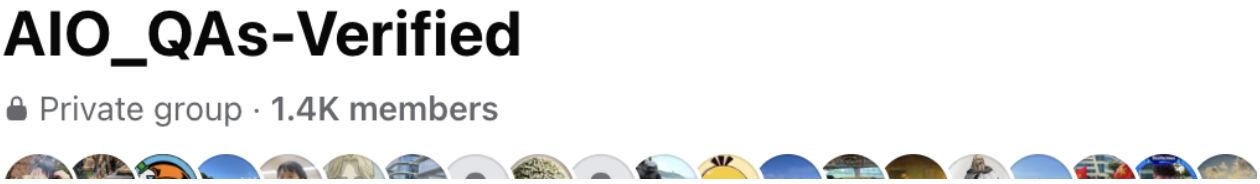
[3] O. M. Parkhi et al., "Cats and dogs", in *2012 IEEE conference on computer vision and pattern recognition*, IEEE, 2012, pp. 3498–3505.

[4] D. Jha et al., "Kvasir-seg: A segmented polyp dataset", in *MultiMedia Modeling: 26th International Conference, MMM 2020*, Springer, 2020, pp. 451–462.

[5] V. Dumoulin et al., "A guide to convolution arithmetic for deep learning", *arXiv preprint arXiv:1603.07285*, 2016.

Phụ lục

1. **Datasets:** Các file dataset được đề cập trong bài có thể được tải tại đây.



Hình 18: Hình ảnh group facebook AIO Q&A.

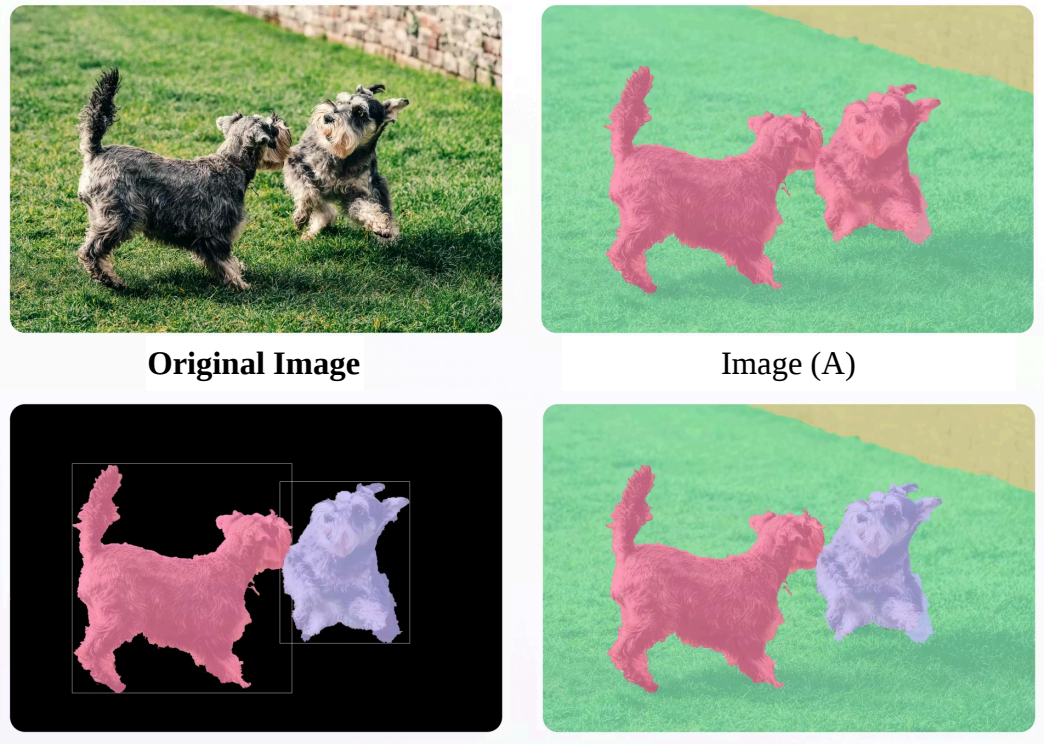
AI VIET NAM
@aivietnam.edu.vn

AI VIET NAM – AI COURSE 2025

Quiz: U-Net Architecture

Trần Hoàng Duy, Nguyễn Phúc Nguyên, Nguyễn Đặng Duy Tân,
Trần Quang Minh và Đậu Văn Nam

1. Dựa trên kiến trúc U-Net và các biến thể cho các tác vụ Dense Prediction, hãy phân tích sự khác biệt giữa các đầu ra trong hình.



Original Image Image (A)

Image (B) Image (C)

Khẳng định nào sau đây là đúng?

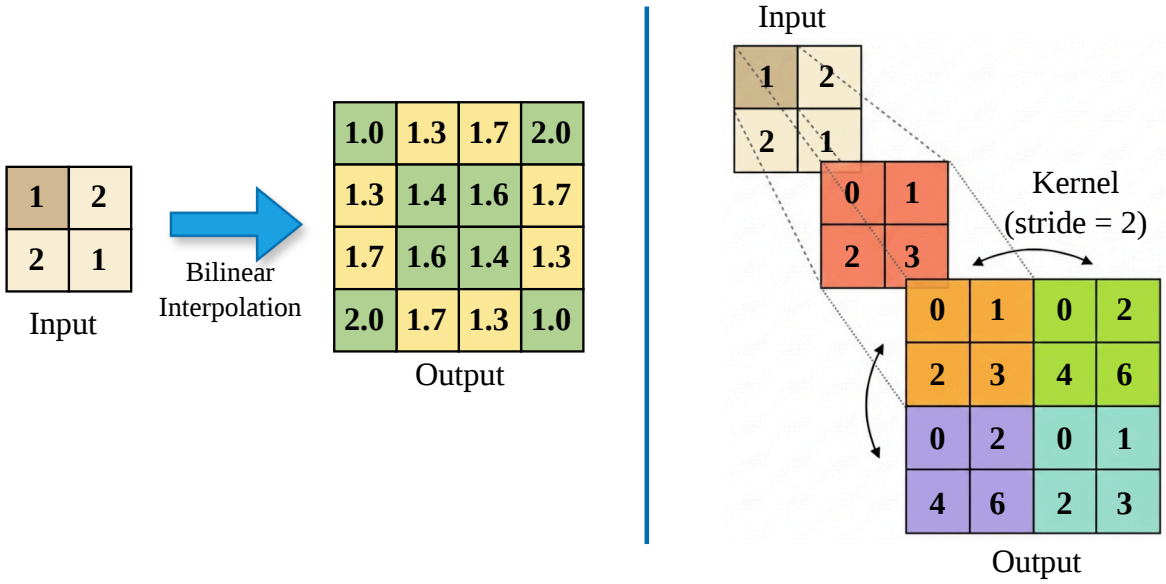
A. Image (A) thực hiện phân biệt từng cá thể đối tượng riêng biệt trong cùng một lớp (class).

B. Image (B) chỉ tập trung vào việc phân loại các điểm ảnh thuộc về nền (background) và vật thể (foreground) nói chung.

C. Image (C) là sự kết hợp giữa việc phân loại lớp của điểm ảnh (Semantic) và phân tách các cá thể đối tượng (Instance).

D. Image (A) và (B) thực chất là một bài toán, chỉ khác nhau về cách hậu xử lý để đổ màu minh họa.

2. Trong kiến trúc U-Net, phần Decoder có nhiệm vụ khôi phục lại độ phân giải ban đầu (upsampling) từ các feature map nhỏ và chứa nhiều ngữ nghĩa. Để phóng đại các feature map này, chúng ta thường phải lựa chọn giữa hai phương pháp chính là Transposed Convolution và Bilinear Interpolation.



Hỏi nhận định nào dưới đây là chính xác về hai phương pháp này?

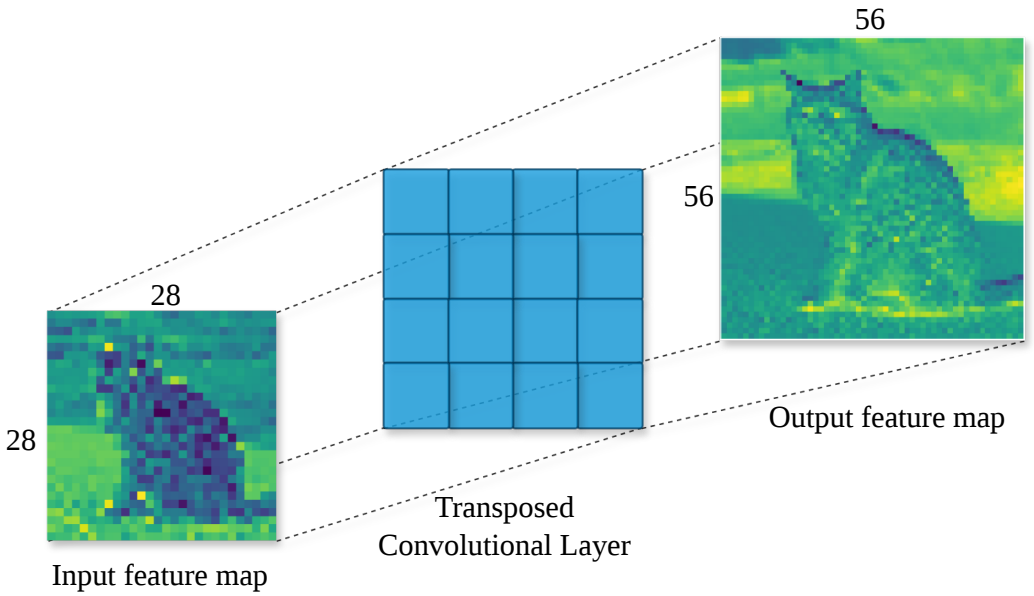
A. Transposed Convolution là phép nội suy cố định, nó không đóng góp vào việc tăng số lượng tham số học tập của mô hình.

B. Transposed Convolution cho phép mạng tự học cách tối ưu hóa việc khôi phục độ phân giải.

C. Bilinear Interpolation thường dẫn đến việc mất thông tin gradient, khiến Decoder không thể hội tụ tốt bằng Transposed Convolution.

D. Cả hai phương pháp đều giải quyết bài toán upsampling thông qua việc chèn thêm các giá trị 0 vào giữa các pixel (zero-padding).

3. Cho một input feature map với kích thước 28×28. Để thực hiện quá trình upsampling, feature map này được đưa qua một lớp Transposed Convolution có các siêu tham số có kernel size $K = 4$ và padding $P = 1$. Cần thiết lập giá trị stride (S) là bao nhiêu để output feature map đạt được kích thước chính xác là 56×56?



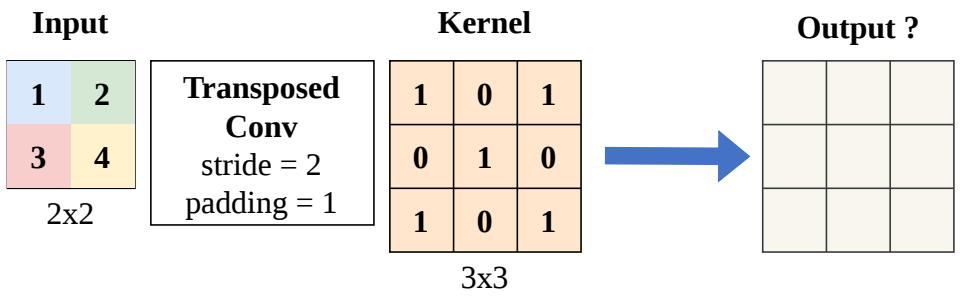
A. $S = 1$.

B. $S = 2$.

C. $S = 3$.

D. $S = 4$.

4. Cho đầu vào feature map X kích thước 2×2 và kernel K kích thước 3×3 như sau:



Thực hiện phép Transposed Convolution với cấu hình (stride = 2, padding = 1). Hãy tính output feature map thu được (Lưu ý: không flip kernel, không dùng bias)?

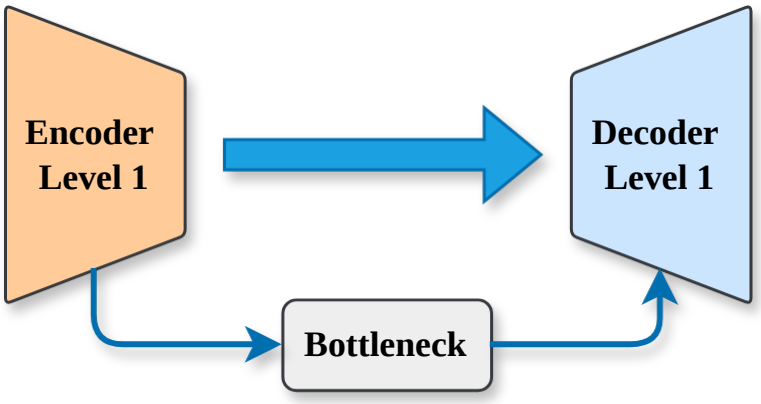
A. $\begin{bmatrix} 1 & 0 & 2 \\ 0 & 10 & 0 \\ 3 & 0 & 4 \end{bmatrix}$.

B. $\begin{bmatrix} 1 & 0 & 3 \\ 0 & 10 & 0 \\ 3 & 0 & 7 \end{bmatrix}$.

C. $\begin{bmatrix} 0 & 1 & 0 \\ 2 & 4 & 2 \\ 0 & 3 & 0 \end{bmatrix}$.

D. $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$.

5. Trong các kiến trúc phân đoạn ảnh trước đây, việc nén dữ liệu liên tục ở Encoder tuy giúp mạng hiểu được ngữ nghĩa (semantic) nhưng lại làm mất đi các chi tiết hình học và vị trí không gian của vật thể. Để khắc phục, U-Net đã sử dụng cơ chế Skip Connections, thay vì bắt nhánh Decoder khôi phục ảnh từ thông tin nhỏ ở Bottleneck, U-Net dẫn trực tiếp các feature map từ nhánh Encoder sang để ghép nối. Điều này giúp mạng định hình chính xác vật thể hơn.



Dựa trên nguyên lý này, hãy xét quá trình xử lý của một mô hình U-Net sau:

- Tại Encoder Level 1: Thu được feature map E_1 có kích thước $48 \times 48 \times 32$.
- Tại Bottleneck: Feature map B bị nén xuống còn $24 \times 24 \times 64$.
- Tại Decoder Level 1: Feature map B đi qua lớp Transposed Convolution với cấu hình: $K = 3$, $S = 2$, $P = 1$, $OP = 1$ và số lượng bộ lọc (filters) giảm xuống còn 32.

Sau bước upsampling, kết quả sẽ được ghép nối dọc theo chiều sâu (channels) với feature map E_1. Hỏi kích thước của khối dữ liệu thu được sau phép ghép nối này là bao nhiêu?

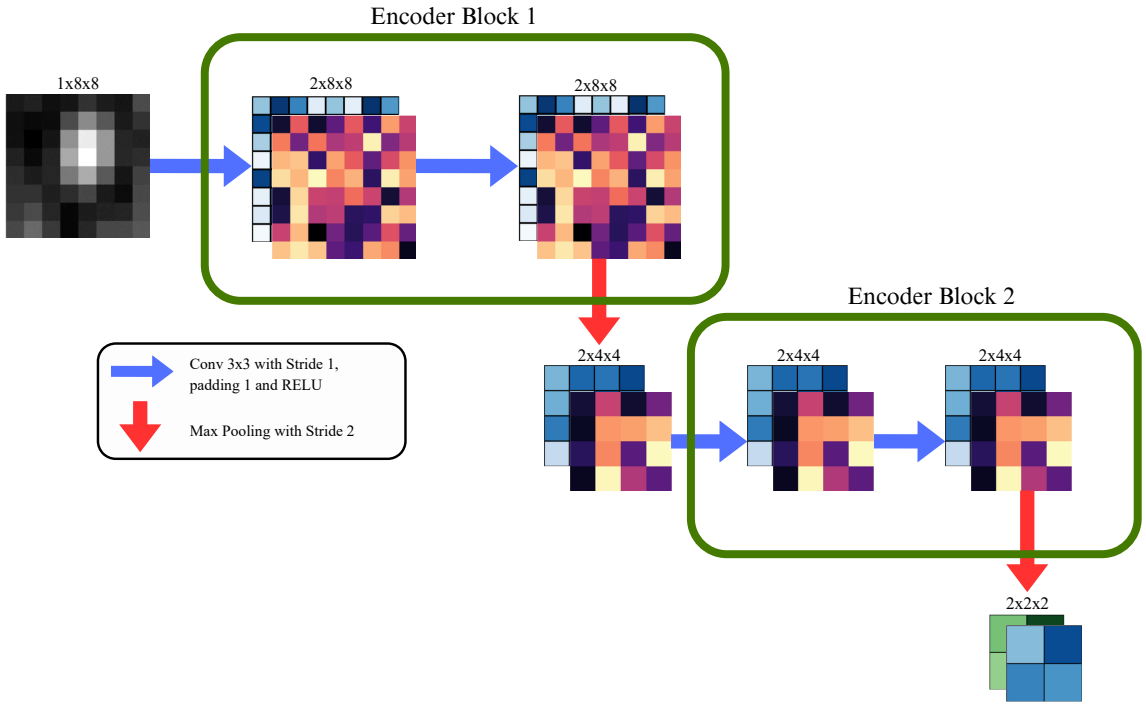
A. $24 \times 24 \times 64$.

B. $48 \times 48 \times 32$.

C. $48 \times 48 \times 64$.

D. $96 \times 96 \times 64$.

6. Trong kiến trúc U-Net, phần Encoder có nhiệm vụ trích xuất đặc trưng và giảm kích thước không gian của ảnh. Cho một ảnh đầu vào kênh đơn (1 channel) kích thước 8×8, được biểu diễn bởi ma trận X. Ảnh này đi qua 2 stages liên tiếp của nhánh Encoder:



Dữ liệu ma trận đầu vào X:

$$X = \begin{bmatrix} 10 & 10 & 10 & 10 & 0 & 0 & 0 & 0 \\ 10 & 10 & 10 & 10 & 0 & 0 & 0 & 0 \\ 10 & 10 & 10 & 10 & 0 & 0 & 0 & 0 \\ 10 & 10 & 10 & 10 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 4 & 4 & 4 & 4 \\ 0 & 0 & 0 & 0 & 4 & 4 & 4 & 4 \\ 0 & 0 & 0 & 0 & 4 & 4 & 4 & 4 \\ 0 & 0 & 0 & 0 & 4 & 4 & 4 & 4 \end{bmatrix}.$$

Tầng 1 (Encoder Block 1): Chuỗi phép toán **Conv 1.1** $\rightarrow$ **ReLU** $\rightarrow$ **Conv 1.2** $\rightarrow$ **ReLU** $\rightarrow$ **MaxPooling**.

- **Conv 1.1 & 1.2:** Đều sử dụng 2 Kernel (3×3), Padding $= 1$, Stride $= 1$.

$$K_{1.1} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad K_{1.2} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

  (Lưu ý: Ở lớp Conv 1.2, giả sử các ma trận trọng số (weight) là giống hệt nhau trên không gian các kênh đầu vào. Tức là ta dùng lại $K_{1.1}$ và $K_{1.2}$ chung cho tất cả các kênh).

- **MaxPooling:** Kích thước 2×2, Stride $= 2$.

Tầng 2 (Encoder Block 2): Chuỗi phép toán **Conv 2.1** $\rightarrow$ **ReLU** $\rightarrow$ **Conv 2.2** $\rightarrow$ **ReLU** $\rightarrow$ **MaxPooling**.

- **Conv 2.1 & 2.2:** Giống nhau và gồm 2 Kernel (3×3), Padding = 1, Stride = 1.

$$K_{2.1} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad K_{2.2} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

  (Lưu ý: Tương tự Tầng 1, giả sử các ma trận trọng số được dùng chung cho tất cả các kênh đầu vào ở cả lớp Conv 2.1 và Conv 2.2).

- **MaxPooling:** Kích thước 2×2, Stride = 2.

Hỏi ma trận đầu ra cuối cùng sau Tầng 2 (Kích thước $2 \times 2 \times 2$) là đáp án nào dưới đây?

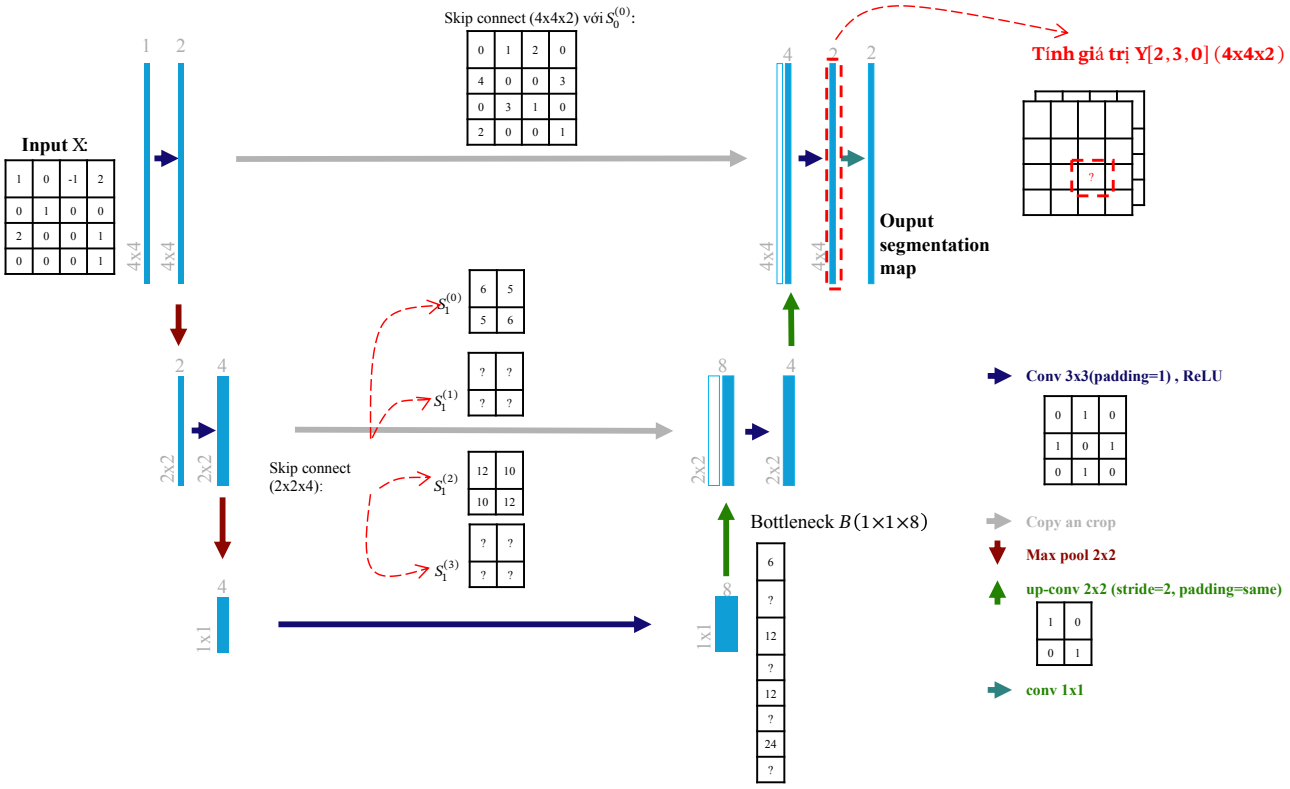
A. Channel 0: $\begin{bmatrix} 40 & 0 \\ 60 & 16 \end{bmatrix}$ Channel 1: $\begin{bmatrix} -20 & 0 \\ 0 & -8 \end{bmatrix}$.

B. Channel 0: $\begin{bmatrix} 20 & 0 \\ 40 & 8 \end{bmatrix}$ Channel 1: $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$.

C. Channel 0: $\begin{bmatrix} 40 & 0 \\ 60 & 16 \end{bmatrix}$ Channel 1: $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$.

D. Channel 0: $\begin{bmatrix} 20 & -10 \\ 40 & 12 \end{bmatrix}$ Channel 1: $\begin{bmatrix} -10 & 0 \\ -4 & -8 \end{bmatrix}$.

7. Xét một mô hình U-Net Tiny như dưới đây:



Sau khi qua khối Encoder, ta thu được các tensor đầu vào cho khối Decoder như sau:

- Skip mức 1 (kênh 0):

$$S_0^{(0)} = \begin{bmatrix} 0 & 1 & 2 & 0 \\ 4 & 0 & 0 & 3 \\ 0 & 3 & 1 & 0 \\ 2 & 0 & 0 & 1 \end{bmatrix}.$$

- Skip mức 2 (các kênh cần thiết):

$$S_1^{(0)} = \begin{bmatrix} 6 & 5 \\ 5 & 6 \end{bmatrix}, \qquad S_1^{(2)} = \begin{bmatrix} 12 & 10 \\ 10 & 12 \end{bmatrix}.$$

- Bottleneck $(1\times1\times8)$:

$$B^{(0)} = 6, \quad B^{(4)} = 12, \quad B^{(2)} = 12, \quad B^{(6)} = 24.$$

Hỏi giá trị tại $Y[2,3,0]$ là bao nhiêu?

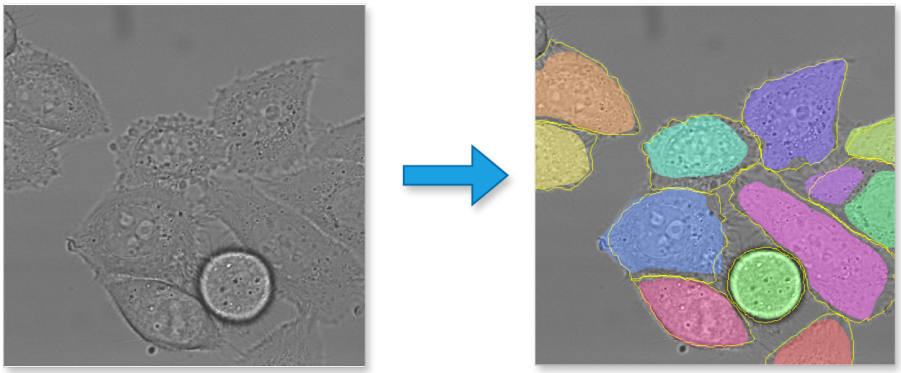
A. 209.

B. 210.

C. 354.

D. 420.

8. Kiến trúc U-Net ban đầu được thiết kế chuyên biệt cho các bài toán phân đoạn ảnh y sinh, nơi các đối tượng như tế bào thường xuất hiện dày đặc và nằm sát nhau. Thử thách lớn nhất ở đây không chỉ là phân loại đúng pixel thuộc về đối tượng nào, mà còn phải tách biệt rạch ròi các thực thể (instances) nằm tiếp giáp, tránh hiện tượng chúng bị dự đoán dính liền thành một khối lớn.



Original Image Segmented Image

Để giải quyết vấn đề này, mạng U-Net đã sử dụng hàm mất mát Weighted Cross Entropy Loss thay vì dùng hàm Standard Cross Entropy Loss thông thường. Mục đích của thiết kế này là gì?

A. Để tăng tốc độ hội tụ của mô hình bằng cách loại bỏ các pixel không quan trọng ở vùng trung tâm đối tượng.

B. Để cân bằng dữ liệu khi số lượng pixel của lớp đối tượng (foreground) ít hơn nhiều so với lớp nền (background).

C. Để yêu cầu mô hình phạt nặng các sai số xảy ra tại đường biên giữa các đối tượng tiếp giáp nhau.

D. Để tối ưu hóa bộ nhớ khi thực hiện phép toán Transposed Convolution ở phần Decoder.

9. Bạn hãy mở notebook U-Net from scratch trong Link để làm bài toán phân đoạn khối u trên ảnh nội soi y tế. Mạng U-Net này tích hợp cơ chế Skip Connections ở Decoder, trong đó feature map sau khi upsample được concatenate với skip feature từ encoder tương ứng. Việc ghép nối theo channel dimension sẽ giúp mô hình khôi phục lại các chi tiết không gian đã mất và nhận diện rõ ranh giới của khối u.

Image (A)

Image (B)

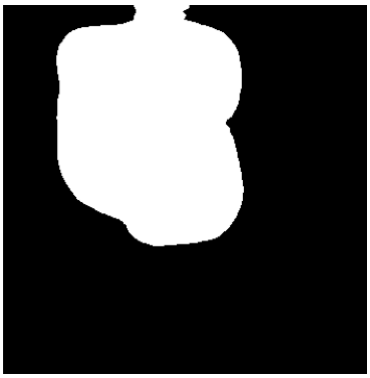


Image (C)

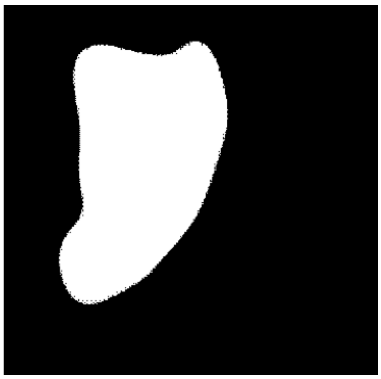


Image (D)

Hãy hoàn thiện class `Decoder` đúng theo yêu cầu trong notebook, sau đó chạy các cell tới phần inference và dùng đúng ảnh đầu vào đã được cung cấp sẵn trong notebook. Đối chiếu vùng dự đoán pred mask của bạn với bốn kết quả minh họa trong hình trên và chọn hình trùng khớp.

A. Image (A).

B. Image (B).

C. Image (C).

D. Image (D).

10. Bạn hãy mở notebook fine-tune SegFormer trong Link. Kiến trúc này được sử dụng thay U-Net nhằm tận dụng thư viện Transformers có sẵn. Trên thực tế Hugging Face vẫn cung cấp nhiều mô hình pre-trained U-Net cho segmentation, nhưng các mô hình này thường yêu cầu việc tự cấu hình phức tạp. Tuy nhiên, sử dụng SegFormer có một số lưu ý về việc xử lý dữ liệu. Khác với U-Net, kiến trúc này sử dụng All-MLP Decoder để giảm khối lượng tính toán. Thiết kế này làm cho feature map dự đoán `logits` ở đầu ra có kích thước nhỏ hơn nhãn gốc `labels`. Để khắc phục sự chênh lệch này cần phải xử lý bất đồng bộ (thường dùng Bilinear Interpolation) ở phần **Evaluate** để phóng to `logits` cho khớp với `labels`.

Image (A)

Image (B)

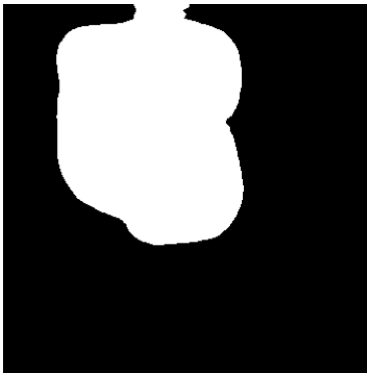


Image (C)

Image (D)

Hãy hoàn thiện đoạn code xử lý chênh lệch, sau đó chạy các cell tới phần inference và dùng đúng ảnh đầu vào đã được cung cấp sẵn trong notebook. Đối chiếu pred mask của bạn với bốn kết quả minh họa trong hình trên và chọn hình trùng khớp.

A. Image (A).

B. Image (B).

C. Image (C).

D. Image (D).

AI VIET NAM
@aivietnam.edu.vn

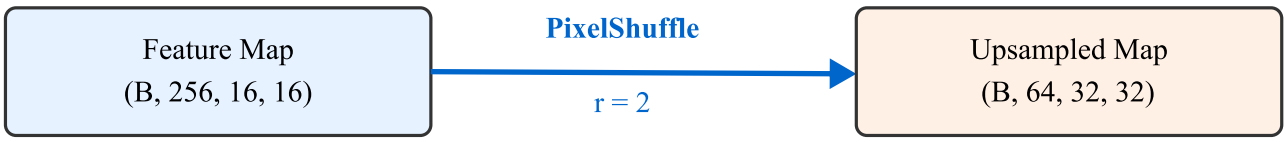
AI VIET NAM COURSE 2025

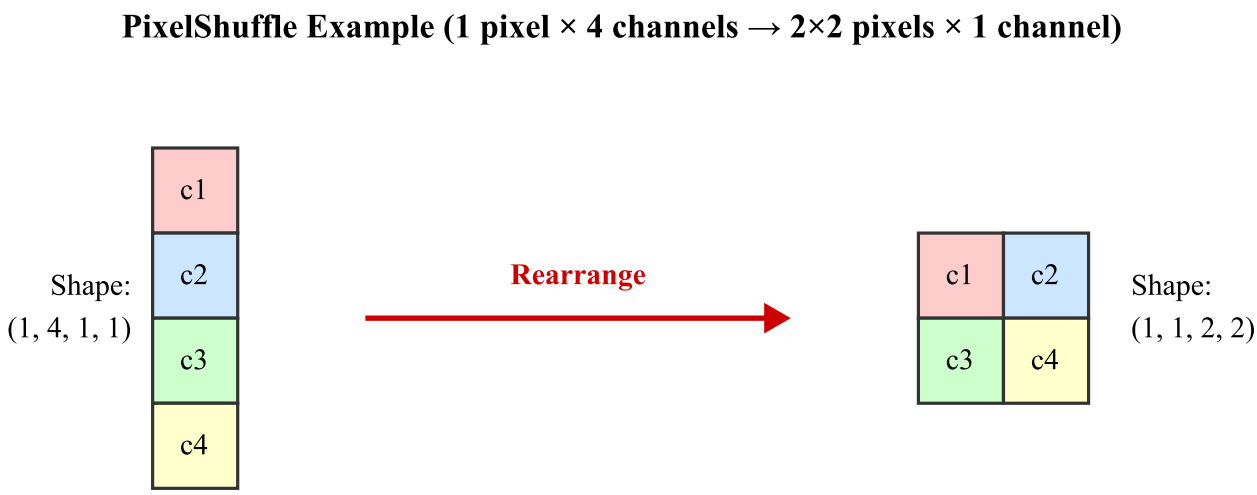
Quiz: Domain Conversion

Nguyễn Thế Hào và Đinh Quang Vinh

I. Đề bài

Câu 1 Quan sát minh hoạ PixelShuffle (tỷ lệ phóng $r = 2$) dùng để tăng resolution của feature map:





Bạn đang có feature map X với shape $(B, 256, 16, 16)$ và muốn upsample lên $(B, 64, 32, 32)$. Phát biểu nào sau đây mô tả **đúng nhất** về PixelShuffle trong bối cảnh decoder của UNet?

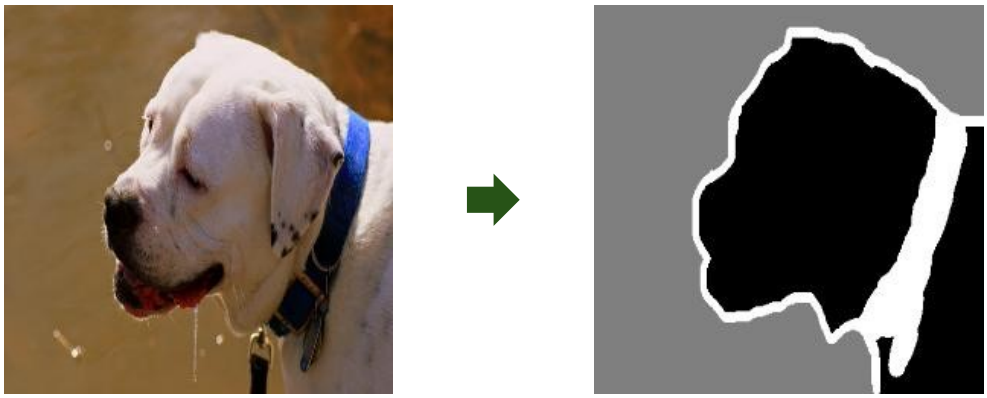
(a) PixelShuffle là phép nội suy (interpolation) nên không cần tăng số kênh trước đó; nó tự tạo thông tin mới bằng cách làm mượt.

(b) PixelShuffle là phép **hoán vị/reshape có cấu trúc** (không tham số học) chuyển $C \cdot r^2$ kênh thành tăng kích thước không gian; do đó **bắt buộc** tensor đầu vào phải có đúng $C \cdot r^2$ kênh.

(c) PixelShuffle tương đương ConvTranspose với stride=2 và kernel=4 nhưng "ổn định hơn" vì có tham số học ít hơn.

(d) PixelShuffle luôn thay thế được skip connection vì nó bảo toàn toàn bộ chi tiết từ encoder.

Câu 2 Mỗi pixel của ảnh (H,W) sẽ được phân loại vào một trong 12 nhóm (cụm) khác nhau. Shape của label map **sau khi lấy argmax theo chiều lớp** là gì?



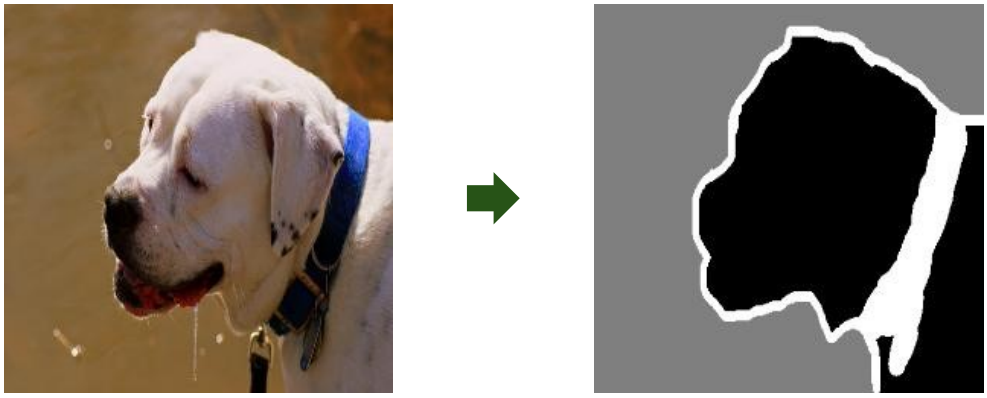
(a) shape=(1,)

(b) shape=(5,)

(c) shape=(H,W)

(d) shape=(5,H,W)

Câu 3 Bài toán phân cụm (segmentation) ảnh có size=(H,W). Giải bài toán theo cách regression có được không?



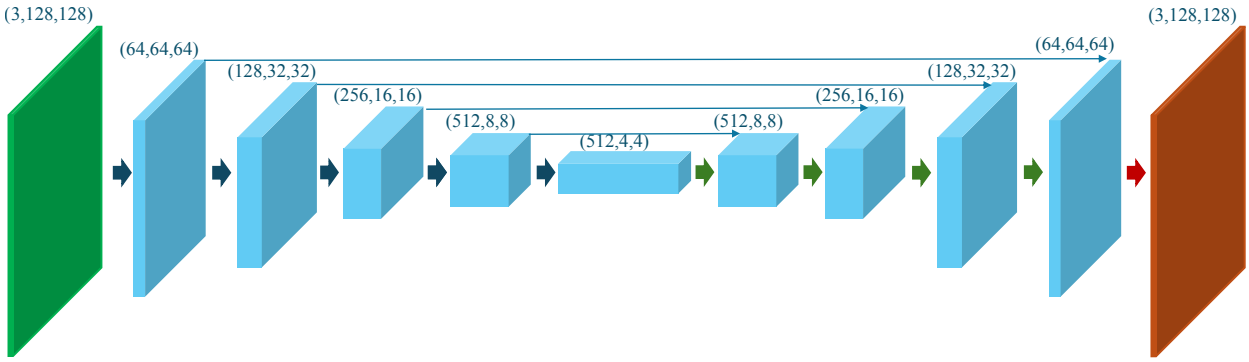
(a) Được

(b) Không được

(c) Không xác định

(d) Không biết

Câu 4 Việc dùng skip-connection như hình dưới làm tăng trọng số của mô hình thế nào?



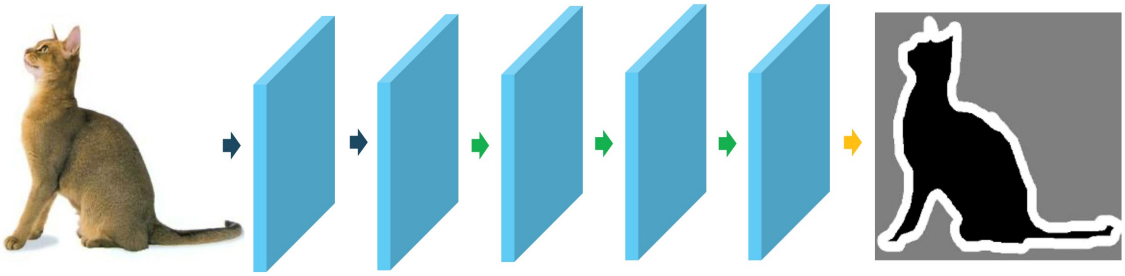
(a) Tăng không đáng kể

(b) Tăng nhiều

(c) Không tăng

(d) Đáp án khác

Câu 5 (Hàm loss sẽ thiết kế sau). Có bao nhiêu model thiết kế đúng cho hình sau?



```
nn.Sequential(
    ...,
    nn.Conv2d(...)
)
```

```
nn.Sequential(
    ...,
    nn.Conv2d(...),
    nn.ReLU()
)
```

```
nn.Sequential(
    ...,
    nn.Conv2d(...),
    nn.PReLU()
)
```

```
nn.Sequential(
    ...,
    nn.Conv2d(...),
    nn.Softmax(...)
)
```

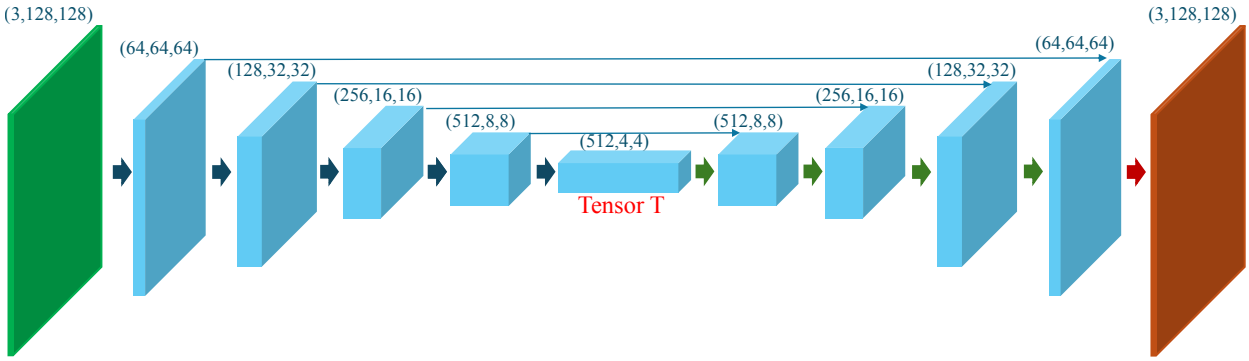
(a) Có 1

(b) Có 2

(c) Có 3

(d) Có 4

Câu 6 Cho thông tin như hình vẽ. Chúng ta có skip-connection (SK) và tensor T. Hãy bắt cặp phù hợp giữa nhóm (SK, T) với (cung cấp thông tin cục bộ local, thông tin toàn cục global]



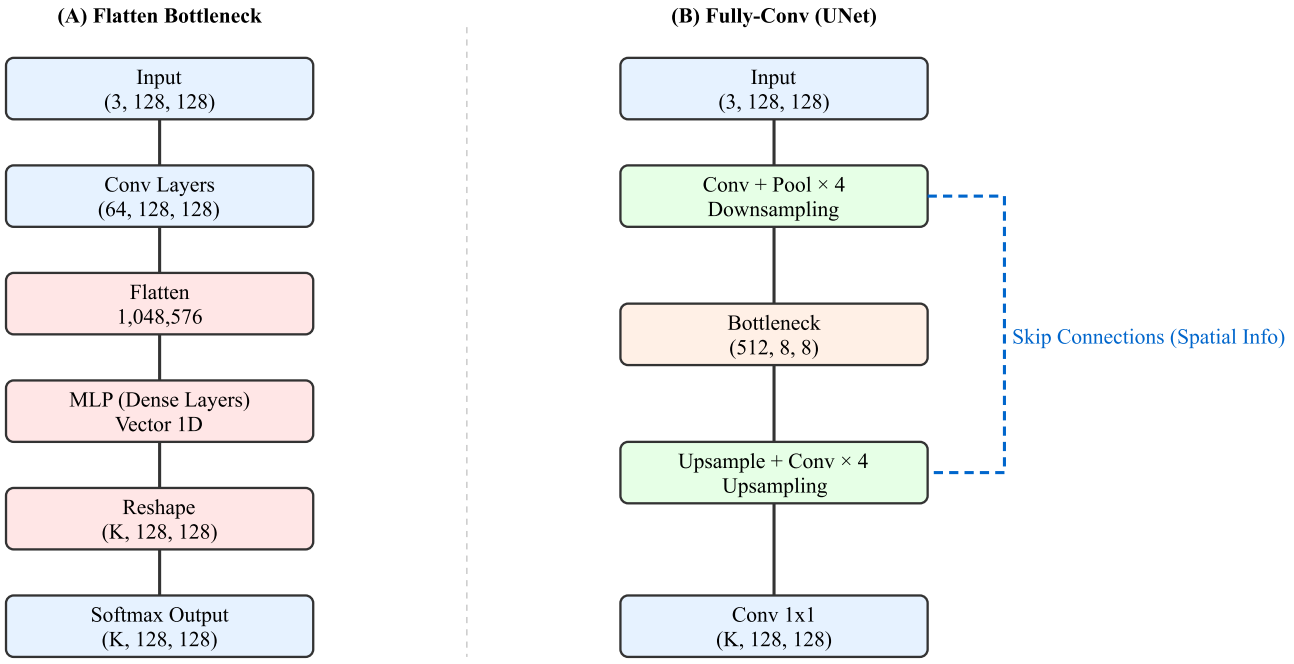
(a) SK–local và T–global

(b) SK–global và T–global

(c) SK–local và T–local

(d) SK–global và T– local

Câu 7 Quan sát hai kiến trúc mô hình bên dưới (A) và (B):



Cả hai mô hình đều có thể tạo ra output tensor cùng kích thước (B, K, H, W) cho segmentation. Tuy nhiên, phát biểu nào sau đây phân biệt **chính xác nhất** lý do tại sao (A) thường thất bại hơn (B) trên dữ liệu mới (dịch chuyển/biến dạng vật thể), dù training loss có thể vẫn giảm?

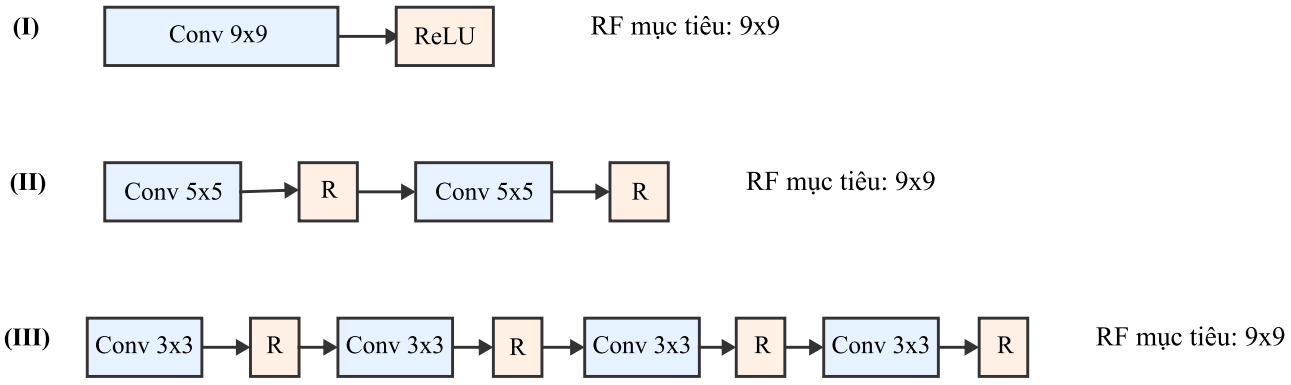
(a) (A) thất bại chủ yếu vì số tham số nhiều hơn (B), gây overfitting; còn về mặt cơ chế xử lý không gian thì hai mô hình là tương đương.

(b) (A) phá vỡ **tính tương đương theo tịnh tiến (translation equivariance)** và **inductive bias không gian** do `Flatten` tạo ánh xạ phụ thuộc vị trí tuyệt đối; (B) giữ tính này nhờ chỉ dùng phép tích chập và sampling theo lưới.

(c) (A) không thể học segmentation vì `Flatten` làm mất hoàn toàn thông tin không gian; do đó output sẽ luôn là mask hằng (constant mask) bất kể input.

(d) (B) luôn tốt hơn (A) vì có MaxPooling giúp tăng độ chính xác; nếu bỏ pooling thì (B) cũng sẽ kém như (A).

Câu 8 Để tăng receptive field trong segmentation, ta xét ba thiết kế khối tích chập (giả sử $C_{in} = C_{out} = 64$, bỏ qua bias):
Mục tiêu là đạt receptive field hiệu dụng **9×9** với **ít tham số nhất** nhưng vẫn giữ khả năng biểu diễn phi tuyến tốt. Chọn phát biểu **đúng nhất**:



** Chú thích: R = ReLU. Các lớp Conv đều giữ nguyên kích thước không gian và số kênh.*

(a) (I) ít tham số nhất vì chỉ có 1 lớp, do đó luôn tối ưu cả compute lẫn khả năng học.

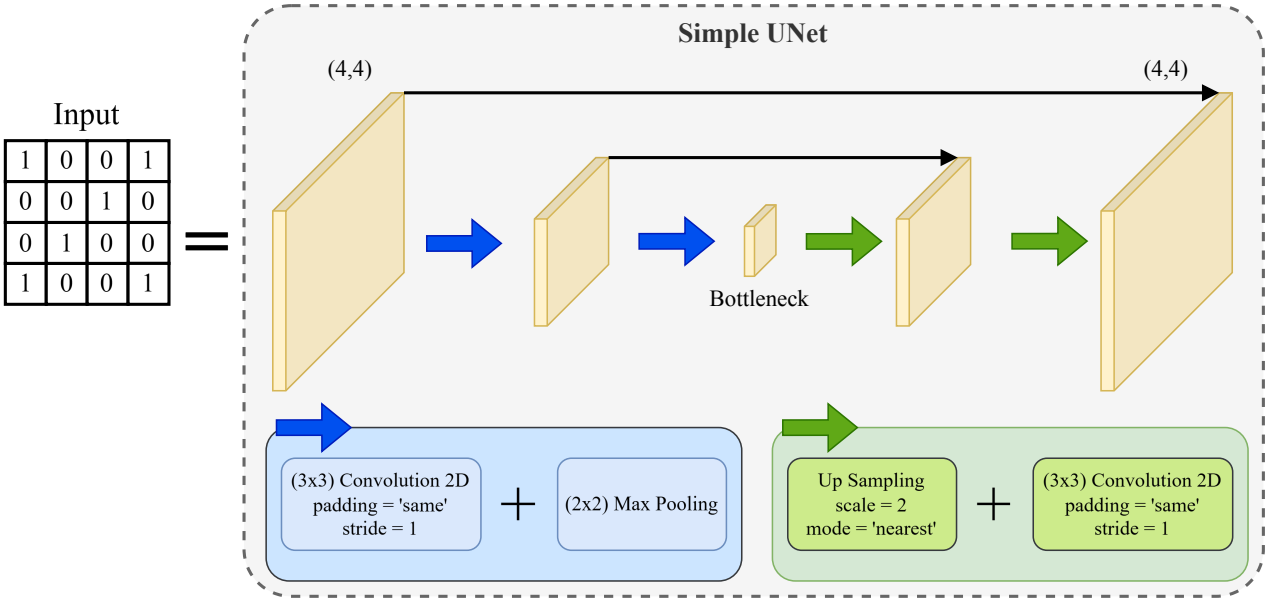
(b) (II) ít tham số nhất và luôn tốt hơn (III) vì kernel 5×5 "nhìn rộng" nên cần ít lớp.

(c) (III) là lựa chọn hợp lý nhất: đạt receptive field 9×9 với ít tham số hơn (I) và (II), đồng thời có nhiều điểm phi tuyến (ReLU) hơn.

(d) Cả (I)(II)(III) có cùng số tham số vì $C_{in} = C_{out}$ giống nhau, chỉ khác số lượng lớp (layers).

Mô tả:

Hình minh họa này sẽ được sử dụng cho câu 9 và câu 10, ta có đầu vào là một ma trận kích thước 4×4 và một mạng **U-Net phiên bản đơn giản** với kiến trúc như sau:



Kiến trúc mô hình U-net đơn giản.

Encoder

Gồm 2 bước *down-sampling*:

- **Conv1:** kernel 3×3, `padding='same'`, toàn bộ trọng số $= 1$, không dùng bias.
- **Max Pooling:** kernel 2×2, stride $= 2$.
- **Conv2:** kernel 3×3, `padding='same'`, toàn bộ trọng số $= 1$, không dùng bias.
- **Max Pooling:** kernel 2×2, stride $= 2$.

Kết quả cuối cùng của bước này là **bottleneck**.

Decoder

Gồm 2 bước *up-sampling*:

- **Upsampling 1:** `scale=2`, `mode='nearest'`.
- **Conv3:** kernel 3×3, `padding='same'`, toàn bộ trọng số $= 1$, không dùng bias.
- **Skip Connection 1:** cộng với đầu ra của **Conv2**.

- **Upsampling 2:** `scale=2`, `mode='nearest'`.
- **Conv4:** kernel 3×3, `padding='same'`, toàn bộ trọng số $= 1$, không dùng bias.
- **Skip Connection 2:** cộng với đầu ra của **Conv1**.

Đầu ra cuối cùng (kích thước 4×4) là **output** của **Conv4**.

Lưu ý: Trong bài này, tất cả các lớp convolution đều sử dụng kernel 3×3 với toàn bộ trọng số bằng 1, không có bias; đồng thời input và output đều có chiều sâu (số kênh) bằng 1.

Question 9: Tính ma trận **bottleneck** sau khi thực hiện 2 lần down-sampling.

(a) `[[3]]`

(b) `[[4]]`

(c) `[[12]]`

(d) `[[16]]`

Question 10: Tính **output cuối cùng** của mạng.

(a)
```
[[241, 362, 362, 242],
[362, 543, 543, 362],
[362, 543, 543, 362],
[242, 362, 362, 241]]
```

(b)
```
[[211, 332, 332, 212],
[332, 513, 513, 332],
[332, 513, 513, 332],
[212, 332, 332, 211]]
```

(c)
```
[[181, 302, 302, 182],
[302, 483, 483, 302],
[302, 483, 483, 302],
[182, 302, 302, 181]]
```

(d)
```
[[151, 272, 272, 152],
[272, 453, 453, 272],
[272, 453, 453, 272],
[152, 272, 272, 151]]
```